



CLOUD CONNECTED AQI NETWORK FOR PAKISTAN

by

Amna Ahmad

Zujaja Ibrahim

Supervisor

Faraan Mahmood

Department of Electrical Engineering

Institute of Space Technology, Islamabad

July 2024



CLOUD CONNECTED AQI NETWORK FOR PAKISTAN

by

Amna Ahmad

Zujaja Ibrahim

Supervisor

Faraan Mahmood

APPROVAL BY BOARD OF EXAMINERS

Supervisor

Co-Supervisor

Internal Examiner

External Examiner

AUTHORS DECLARATION

I take full responsibility of the research work conducted during the thesis titled **“Cloud Connected AQI Network for Pakistan”**.

I solemnly declare that the research and development work presented in the thesis is done solemnly by me with no significant help from any other person; however, small help wherever taken is duly acknowledged. I have also written the thesis by myself. Moreover, I have not presented this thesis or any part of the thesis previously to any other degree awarding institution within Pakistan or abroad.

I understand that the management of IST has a zero-tolerance policy towards plagiarism. Therefore, I as an author of the above-mentioned thesis solemnly declare that no portion of my thesis has been plagiarized and any material used in this Thesis does not contain any literal citing (verbatim) of more than 70 words (total) even by giving a reference unless I have obtained the written permission of the publisher to do so. Furthermore, the work presented in the thesis is my own original work and I have positively cited the related work of the other researchers by clearly differentiating my work from their relevant work.

I further understand that if I am found guilty of any form of plagiarism in my thesis work even after my graduation, the institute reserves the right to revoke my **BS** degree. Moreover, the Institute will also have the right to publish my name on its website that keeps a record of the students who plagiarized in their thesis work.

Amna Ahmad

Zujaja Ibrahim

I hereby acknowledge that submitted thesis is final version and should be scrutinized for plagiarism as per IST policy

Faraan Mahmood

Dated: _____

Verified by Plagiarism Cell Officer

Dated_____

CERTIFICATE

This is to certify that the research work described in this thesis is the original work of the author and has been carried out under my direct supervision. I have personally gone through all the data/results/materials reported in the manuscript and certify their correctness/authenticity. I further certify that the material included in this thesis is not plagiarized and has not been used in part or full in a manuscript already submitted or in the process of submission in partial/complete fulfillment of the award of any other degree from any institution. I also certify that the thesis has been prepared under my supervision according to the prescribed format and I endorse its evaluation for the award of Bachelor of Science in Electrical Engineering degree through the official procedures of the Institute.

Faraan Mahmood

Copyright © 2022

This document is jointly copyrighted by the authors and the Institute of Space Technology (IST). Both authors and IST can use, publish or reproduce this document in any form. Under the copyright law no part of this document can be reproduced by anyone, except copyright holders, without the permission of authors.

DEDICATION

This research work is dedicated to our beloved Parents and Teachers and Friends who supported and motivated us in tough times and made us into what we are today.

ABSTRACT

The escalating concerns over air quality, particularly in urban areas like Islamabad, necessitate effective remote monitoring and alert systems. This document details the development of a prototype network for monitoring the Air Quality Index (AQI) in Islamabad. The project employs strategically placed nodes with calibrated sensors, transmitting real-time data to a central server through the MQTT protocol and sending hourly SMS alerts to the user in case of the absence of a Wi-Fi connection. The data obtained through this network was used in predictive analysis using machine learning algorithms, including Linear Regression, Cat-Boost Regressor, Random Forest and Gradient Boost Regressor. A comparative analysis was conducted to determine the best-fit model for accurate AQI predictions.

1.1.1.1 TABLE OF CONTENTS

2	CHAPTER 01	1
2.1	Introduction	1
2.2	Problem Statement	2
2.3	Objective	3
2.4	Literature Survey.....	3
3	Chapter 02.....	6
3.1	Solution Statement	6
3.2	Individual Description of Equipment used in each node	7
3.2.1	ESP32.....	7
3.2.2	MQ7	8
3.2.3	MQ135	9
3.2.4	DSM501A	10
3.2.5	OLED	11
3.2.6	SIM800L	12
3.2.7	Power Source.....	13
3.2.8	Micro USB cable.....	13
3.3	Block Diagram	14
4	CHAPTER 3: SYSTEM DETAILS.....	15

4.1	Sensors and their calibration:	15
4.1.1	Calibrating MQ-7:	15
4.1.2	Calibrating MQ-135	17
4.1.3	Calibrating DSM501A	17
4.2	Calculating time weighted AQI.....	18
4.3	Using deep sleep mode of ESP32:	20
4.4	OLED display.....	21
4.5	Sending SMS alerts to users.....	22
4.6	Final circuit designing:.....	24
4.7	PCB designing:.....	25
4.8	CAD 3D Casing designing:.....	28
4.9	Cloud Setup	30
4.10	Dataset:	31
4.11	Process for predictive analysis:	32
4.11.1	Data Visualization:.....	33
4.11.2	Parameter correlation with AQI:	33
4.11.3	Training Linear Regression model:.....	33
4.11.4	Training Random Forest model:	33
4.11.5	Training Gradient boosting model:	34
4.11.6	Training CatRegressor model:	34

4.11.7	Model Evaluation Tests:.....	34
4.11.7.1	Training Time:.....	34
4.11.7.2	Training Score:.....	34
4.11.7.3	Test Score:.....	35
4.11.7.4	R2 Score:.....	35
4.11.7.5	RMSE (Root Mean Square):	35
4.11.7.6	MAPE (Mean Absolute Percentage Error):	35
5	Chapter 4: CODE EXPLANTIONS.....	36
5.1	C programming on ESP32.....	36
5.1.1	Wi-Fi declarations and credentials.....	36
5.1.2	Global Variables.....	37
5.1.3	OLED declarations.....	37
5.1.4	Sleep Mode Declarations	38
5.1.5	GPIO pin definitions and necessary variables' initialization	38
5.1.6	Individual pollutant concentration functions' declaration	39
5.1.7	RTC data array for 24 values	40
5.1.8	Circular Queue Functions.....	40
5.1.9	OLED Display Function.....	42
5.1.10	Function for sending sensor data to cloud.....	43
5.1.11	VoidSetup ().....	44

5.1.12	Blynk.run function for delay	48
5.1.13	PM1 and PM2.5 data function.....	49
5.1.14	CO sensor ppm function.....	50
5.1.15	SO2 sensor ppm Function	51
5.1.16	Air Quality standard mapping conditions	51
5.1.17	AQI calculator function for PM2.5	52
5.1.18	AQI Calculator Function for PM1	53
5.1.19	AQI Calculator function for CO	54
5.1.20	AQI calculator for SO2	55
5.1.21	Weighted Average Calculator Function.....	56
5.2	Predictive analysis:.....	58
5.2.1	Data visualization:.....	58
5.2.2	Data Correlation:	59
5.2.3	Data Splitting;	60
5.2.4	Linear Regression:.....	61
5.2.5	Random Forest model:	62
5.2.6	Gradient Boost Regressor:	63
5.2.7	CAT Boost Regressor:	64
6	CHapter 05: RESULTS.....	66
6.1	Arduino Serial Monitor	66

6.2	Cloud Display Results	69
6.3	Predictive Analysis:.....	70
6.3.1	Data visualization:.....	70
6.3.2	Data Correlation:	73
6.3.3	Data splitting:	74
6.3.4	Predictive modal results:	75
6.3.5	Model test score comparisons:	77
7	Chapter 06.....	78
7.1	UN-SDGs mapping	78
7.1.1	Goal 3: Good Health and Well-being.....	78
7.1.2	Goal 11: Sustainable Cities and Communities.....	78
7.1.3	Goal 13: Climate Action	78
7.1.4	Goal 17: Partnerships for the Goals	78
8	Future work.....	80
9	References.....	81

LIST OF FIGURES

Figure 1: ESP32 micro-controller	8
Figure 2: MQ7 Sensor pinout.....	9
Figure 3: MQ135 sensor pinout	9
Figure 4: DSM501A image	10
Figure 5: SSD1306 image	11
Figure 6: TFT 128X64 image	12
Figure 7: SIM800L image.....	12
Figure 8: 5V 2A Power bank	13
Figure 9: Micro USB cable	13
Figure 10: Block Diagram Illustration	14
Figure 11: Sensitivity characteristics of MQ7	16
Figure 12: Sensitivity characteristics of MQ135	17
Figure 13: Sensor characteristic vs Low ratio	18
Figure 14: EPA's breakpoint and AQI Index [2].....	19
Figure 15: Sleep Mode comparison chart [3].....	20

Figure 16: TFT display.....	22
Figure 17: SSD1306 displaying individual pollutant concentrations and the Final AQI	22
Figure 18: SMS Alerts received on user's phone	23
Figure 19: Node A Circuit Schematic	24
Figure 20: Node B Circuit Schematic	24
Figure 21: Circuit Design of node in proteus.....	26
Figure 22: PCB design of node A	27
Figure 23: PCB Design of Node B.....	27
Figure 24: 3D Model of Node Casing.....	28
Figure 25: Side View and dimensions of Node's casing.....	29
Figure 26: Top View of Node Casing	29
Figure 27: Cloud Network Structure.....	30
Figure 28: Data Visualization on Cloud.....	30
Figure 29: Collected Dataset in CSV format	32
Figure 30: Wi-Fi Declarations	36
Figure 31: Global variables for Cloud data transfer and storage	37

Figure 32: OLED declarations	37
Figure 33: Sleep Mode declarations.....	38
Figure 34: GPIO pins and variables declaration (part 1)	38
Figure 35: GPIO pins and variable declaration (part 2).....	39
Figure 36: Individual Pollutant concentration functions' declaration	39
Figure 37: RTC data array for 24 values.....	40
Figure 38: Queuing Function definition.....	41
Figure 39: Queuing display function.....	42
Figure 40: OLED display function definition	43
Figure 41: Function for sending data to Cloud	44
Figure 42:Void Setup (part 1)	45
Figure 43: Void Setup (part 2)	45
Figure 44: Void Setup (part 3)	46
Figure 45: Void Setup (part 4)	46
Figure 46: Void Setup (part 5)	47
Figure 47: Void Setup (part 6)	47

Figure 48: Void Setup (part 7)	48
Figure 49: Execution time function for Blynk.run() command	49
Figure 50: PM1 and PM2.5 data function	49
Figure 51: CO concentration (ppm) function.....	50
Figure 52: SO2 concentration ppm function.....	51
Figure 53: AQI mapping function.....	52
Figure 54: PM 2.5's AQI calculator function	53
Figure 55: PM1's AQI calculator function	54
Figure 56: CO's AQI calculator function	55
Figure 57: SO2 's AQI calculator function.....	56
Figure 58: Weighted Average function (part 1).....	57
Figure 59: Weighted Average function (part 2).....	58
Figure 60: Data visualization code (part 1).....	58
Figure 61: Data Visualization code (part 2).....	59
Figure 62: Data visualization code (part 3).....	59
Figure 63: Data correlation code.....	59

Figure 64: Code to split dataset.....	60
Figure 65: Linear Regression code	61
Figure 66: Random Forest Model code.....	62
Figure 67: Gradient Boost Regressor code	63
Figure 68: CAT Boost Regressor Model code	64
Figure 69: Serial Monitor display (part 1)	66
Figure 70: Serial Monitor display (part 2)	66
Figure 71: Serial Monitor display (part 3)	67
Figure 72: Serial Monitor display (part 4)	67
Figure 73: Serial Monitor display (part 5)	68
Figure 74: Serial Monitor display (part 6)	69
Figure 75: Cloud data visualization in templates	70
Figure 76: Data Visualization (part 1)	70
Figure 77: Data visualization results (part 2)	71
Figure 78: Data visualization results (part 3)	72
Figure 79: correlation heatmap results	73

Figure 80: Data Split results.....	74
Figure 81: Linear Regression model results.....	75
Figure 82: Random Forest Regression model results	75
Figure 83: Gradient Boost Regressor Model results	76
Figure 84: CAT Boost Regressor Model Results.....	76

LIST OF ABBREVIATIONS AND SYMBOLS

AQI	Air Quality Index
PPM	Parts Per Million
AWS	Amazon Web Services
MQTT	Message Queuing Telemetry Transfer
PM	Particle Matter
PCB	Printed Circuit Board
CO	Carbon mono-Oxide
IoT	Internet of Things

2 CHAPTER 01

2.1 Introduction

In recent years, the escalating concerns surrounding air quality have become a focal point of concern. Urban areas face several challenges in maintaining breathable and sustainable atmospheres. Islamabad, the capital city of Pakistan, is no exception to this predicament, with its ever-growing population and rapid urbanization intensifying the need for effective air quality management.

This paper delves into an initiative undertaken to address the critical issue of air quality in Islamabad. The project centers around the establishment of a Network for Monitoring Atmospheric Parameters and Air Quality Index (AQI). Such a network provides real-time data that not only assesses the prevailing atmospheric conditions but also facilitates informed decision-making for future development.

The network employs strategically placed nodes equipped with calibrated sensors, which monitor key atmospheric parameters and AQI metrics. These sensors transmit real-time data to a central server through the MQTT protocol, ensuring continuous monitoring and analysis. In the event of a Wi-Fi connection failure, the system sends hourly SMS alerts to users, maintaining a consistent flow of information.

Furthermore, the data collected by the network is utilized for predictive analysis using a variety of machine learning algorithms, including Linear Regression, CatBoost Regressor, Random Forest, and Gradient Boost Regressor. This approach enables the development of predictive models that can forecast future air quality trends. A comparative analysis was

conducted to determine the most accurate and reliable model for AQI prediction, thereby enhancing the system's capability to provide actionable insights.

To support this initiative, an AWS database was used for three months of data collection, ensuring reliable and scalable storage of the vast amount of data generated. For visualization of AQI parameters, the Blynk IoT platform was employed, providing an intuitive and user-friendly interface for real-time monitoring and analysis of air quality metrics.

The initiative represents a significant step towards leveraging technology for environmental sustainability and public health in Islamabad and then for Pakistan.

2.2 Problem Statement

Air quality degradation is a growing concern worldwide, particularly in rapidly urbanizing cities like Islamabad, Pakistan. The city's increasing population and industrial activities have led to significant air pollution, posing severe health risks to residents and challenging the sustainability of the urban environment. Despite the critical need for effective air quality management, there is a lack of real-time monitoring systems that can provide accurate and actionable data on changing AQI levels.

Existing air quality monitoring solutions often suffer from absence of Real time data monitors, lack of AQI data for accuracy comparison, and inadequate predictive capabilities, hindering timely interventions and informed decision-making. Consequently, there is an urgent need for a comprehensive, real-time monitoring network that can assess air quality accurately and facilitate timely measures to reduce pollution. Without such a system, the ability to protect public health and manage environmental quality in Islamabad remains severely compromised.

2.3 Objective

Due to rapid increase in pollution, Islamabad is in dire need of a proper air quality monitoring network. Thus, the goal of this project is to cover the process of setting up a prototype for an air quality monitoring network. By providing timely and accurate information, this project not only addresses the immediate challenges of air quality but also lays the groundwork for accurate and informed decision-making.

2.4 Literature Survey

To gain a comprehensive understanding of the current state and advancements in the field of real-time air quality monitoring and predictive analysis, it is essential to review related works. Numerous research studies and projects have been conducted over the past few years, focusing on the development and implementation of air quality monitoring systems. These studies highlight the importance of real-time data collection, data preprocessing, and the application of machine learning models for accurate air quality predictions.

In 2019, Rawal presented a research paper titled "Air Quality Monitoring System," which leverages sensor networks to collect real-time data on pollutants. The system integrates gas sensors, such as MQ135 and MQ7, to measure critical pollutants like carbon monoxide (CO), nitrogen dioxide (NO₂), and particulate matter (PM). By continuously monitoring these parameters, the system ensures accurate and timely information for decision-makers and the public. Rawal's work emphasizes the importance of data preprocessing, which involves cleaning and transforming the raw sensor data to enhance its quality and usability. Additionally, the study highlights the role of machine learning models in predicting air quality trends. By applying algorithms to the preprocessed data, the system

can forecast future air quality levels, enabling timely measures to be taken to protect public health. This research underscores the potential of integrating sensor networks and machine learning for effective air quality management.

Another significant contribution to the field is the research paper by Banerjee, Sikdar, and Roy, titled "IoT-based Air Pollution Monitoring." This study focuses on enhancing environmental awareness through the deployment of a network of sensors across urban areas to collect pollutant data. The data is then transmitted to a central server for analysis. The project emphasizes several key aspects, including real-time monitoring, data visualization, and public access. Continuous data collection from the deployed sensors ensures up-to-date information on air quality. The collected data is visualized to provide clear and comprehensible information to users, aiding in better understanding and awareness. Furthermore, the system ensures that air quality information is accessible to the public, promoting transparency and informed decision-making. By integrating IoT technologies, Banerjee et al. contribute to sustainable urban development. Their work demonstrates how real-time monitoring and data visualization can be leveraged to inform policy decisions and engage the public in environmental protection efforts.

The Islamabad Air Quality Index (AQI) website by IQAir is another noteworthy example of real-time air quality monitoring. This platform provides a comprehensive range of features, including real-time data validation, pollutant concentrations, health recommendations, forecasts, historical data, and opportunities for citizen participation. Key features of the IQAir AQI website include real-time data validation, ensuring the accuracy and reliability of the data presented to users; providing detailed information on the levels of various pollutants in the air; offering advice on how to mitigate health risks associated with poor air quality; predicting future air quality trends to enable timely actions; allowing users

to analyze past air quality patterns; and encouraging public involvement in monitoring and reporting air quality issues. Leveraging these insights, the IQAir platform empowers users with accurate information and contributes to a healthier environment. The integration of real-time monitoring with user-friendly interfaces and actionable recommendations exemplifies the potential of technology in addressing air quality challenges.

These studies and projects provide valuable insights into the development and implementation of real-time air quality monitoring systems. By examining the methodologies and findings of these works, we can identify best practices and potential areas for improvement in our own project. The integration of sensor networks, data preprocessing, machine learning models, and IoT technologies has proven to be effective in enhancing air quality monitoring and predictive analysis.

3 CHAPTER 02

3.1 Solution Statement

Addressing the challenges highlighted in the problem statement, our solution focuses on developing a robust Network for Monitoring Atmospheric Parameters and Air Quality Index (AQI) in Islamabad. This initiative aims to establish a comprehensive real-time monitoring system that effectively assesses and manages air quality in urban environments.

The solution involves deploying strategically placed sensor nodes equipped with calibrated sensors to measure key atmospheric parameters and pollutants such as carbon monoxide (CO), nitrogen dioxide (NO₂), particulate matter (PM), and others. These sensors will continuously collect real-time data and the esp32 microcontroller will process the AQI using the set international standards and a Rolling Average of 24 values which were hourly collected throughout the day is implemented in the final AQI calculation. The AQI along with the individual concentration of the pollutants is transmitted to a central server using the MQTT protocol. In case of Wi-Fi connection failure, the system will still send hourly SMS alerts to users, ensuring uninterrupted monitoring and timely responsiveness.

To enhance the domain and usability of our cloud connected AQI network, our solution incorporates robust data preprocessing techniques and Time Series predictive analysis. This includes cleaning and transforming raw sensor data to eliminate noise and improve data quality. Machine learning algorithms, including Linear Regression, CatBoost Regressor, Random Forest, and Gradient Boost Regressor, will be employed for predictive analysis. This analytical approach enables the development of models capable of forecasting

future air quality trends, thereby facilitating proactive measures to reduce pollution and protect public health.

Furthermore, to optimize energy usage and extend operational longevity, each sensor node's battery is optimized. The ESP32 microcontroller, utilized in each node, enters sleep mode after every hourly reading over a 24-hour period while its RTC memory keeps a record of previous data. This power-saving strategy ensures that approximately 97.5% of the battery life is conserved, enhancing sustainability and reducing operational costs.

The solution also leverages advanced data storage capabilities, utilizing an AWS database for three months of data collection. For visualization and user interface, the Blynk IoT platform is utilized, providing intuitive access to real-time AQI parameters and historical trends. This user-friendly interface empowers stakeholders with actionable insights, promoting informed decision-making and community engagement in environmental management.

By integrating these components into a cohesive network, our solution aims to contribute to sustainable urban development and improve air quality management practices in Islamabad.

3.2 Individual Description of Equipment used in each node

3.2.1 ESP32

The ESP32 microcontroller was chosen for its flexibility and efficiency in IoT applications. Its dual-core architecture, low power consumption, and built-in Wi-Fi

capabilities make it ideal for collecting, processing, and transmitting real-time air quality data. Additionally, the ESP32's ability to enter sleep mode between hourly readings maximizes battery life, crucial for prolonged field deployments without compromising data reliability or operational lifespan.

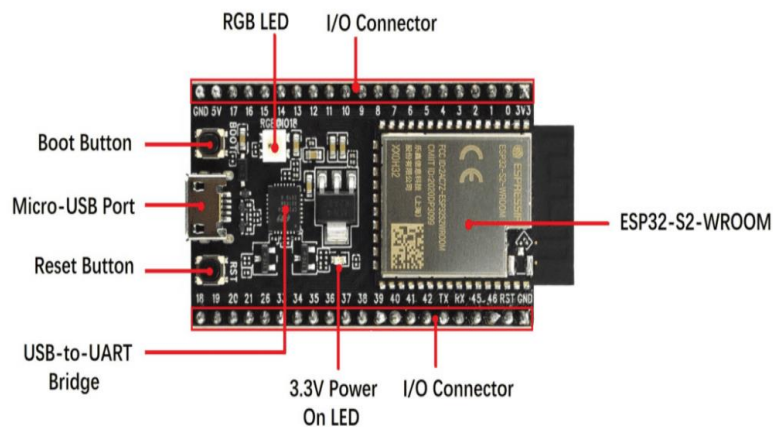


Figure 1: ESP32 micro-controller

3.2.2 MQ7

The MQ7 sensor is commonly used for detecting carbon monoxide (CO) gas in the atmosphere. It operates based on the principle of chemical reaction with CO, generating an electrical signal proportional to the gas concentration. This sensor is valued for its sensitivity and reliability in monitoring CO levels, making it suitable for applications in air quality monitoring and safety systems.



Figure 2: MQ7 Sensor pinout

3.2.3 MQ135

The MQ135 sensor detects sulfur dioxide (SO_2) by utilizing a chemical reaction mechanism. It operates by exposing a tin dioxide (SnO_2) sensing element to the air containing SO_2 . When SO_2 molecules come into contact with the SnO_2 surface, they undergo a chemical reaction, changing the electrical conductivity of the sensor. This change in conductivity is then converted into an electrical signal, which is proportional to the concentration of SO_2 in the air.

Due to its sensitivity to SO_2 , the MQ135 sensor is commonly used in environmental monitoring applications, and indoor air quality assessments where monitoring SO_2 levels is critical for health and safety reasons.

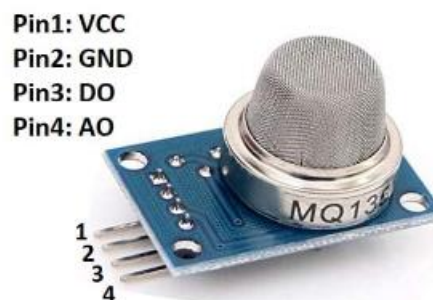


Figure 3: MQ135 sensor pinout

3.2.4 DSM501A

The DSM501A sensor is used for detecting fine particulate matter (PM2.5 and PM10) in the air. It operates based on a principle involving light scattering by particles passing through an LED and photodetector pair. As particles pass through the sensor, they scatter light, and the intensity of scattered light detected by the photodetector correlates with the concentration of particulate matter in the air.

This sensor is valued for its accuracy in measuring PM2.5 and PM10 levels, making it suitable for applications in air quality monitoring systems, indoor air quality assessments, and environmental monitoring. Its ability to provide real-time data on particulate pollution helps in assessing health risks associated with fine particulate matter exposure.



Figure 4: DSM501A image

3.2.5 OLED

The following two types of OLEDs were utilized in this project to show the versatility of display on hardware.

3.2.5.1 SSD1306 128x64

The SSD1306 128x32 OLED display is a small screen known for its clear and bright display. It uses organic light-emitting diodes to show sharp text and graphics without needing a backlight, making it great for small devices and projects where space and power are limited which is the case in this prototype. It typically interfaces with microcontrollers using the I2C communication protocol, allowing for easy and efficient data transfer.

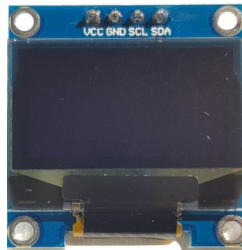


Figure 5: SSD1306 image

3.2.5.2 TFT 128x160

The TFT 128x160 display is a thin-film transistor screen that offers bright colors and high resolution. It provides clear and detailed images, making it suitable for applications that need vibrant display (Creative and enlarged display of AQI). This display usually interfaces with microcontrollers using the SPI communication protocol, ensuring fast and reliable data communication.



Figure 6: TFT 128X64 image

3.2.6 SIM800L

The SIM800L module is a compact GSM/GPRS module used for wireless communication. It enables devices to send SMS alerts and make voice calls over the GSM 2G network. In this project, the SIM800L is utilized to send hourly SMS alerts so the user stays aware of the AQI even when there is no Wi-Fi, ensuring continuous monitoring and timely notifications. Its small size, low power consumption, and reliable connectivity make it ideal for remote monitoring applications.

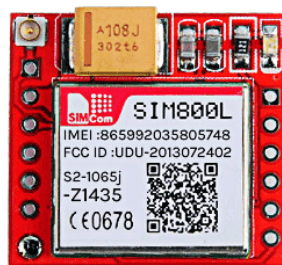


Figure 7: SIM800L image

3.2.7 Power Source

A 5V 2A power bank is used to power the AQI node, providing a reliable and portable power source. The power bank's 5V output is compatible with the node's requirements, and its 2A current rating ensures stable and adequate power for all connected components, enhancing the system's overall reliability and mobility.



Figure 8: 5V 2A Power bank

3.2.8 Micro USB cable

The cable is used to connect the esp32 and to connect the power bank to it, ensuring a steady and reliable power supply. The cable supports the power needed by the AQI node, helping to keep the system running smoothly and reliably in different settings.



Figure 9: Micro USB cable

3.3 Block Diagram

The following flow diagram illustrates the logical working of a single node. The block Diagram looks like a cycle which repeats because ESP32 repeats this process every time after it wakes up from its deep sleep mode.

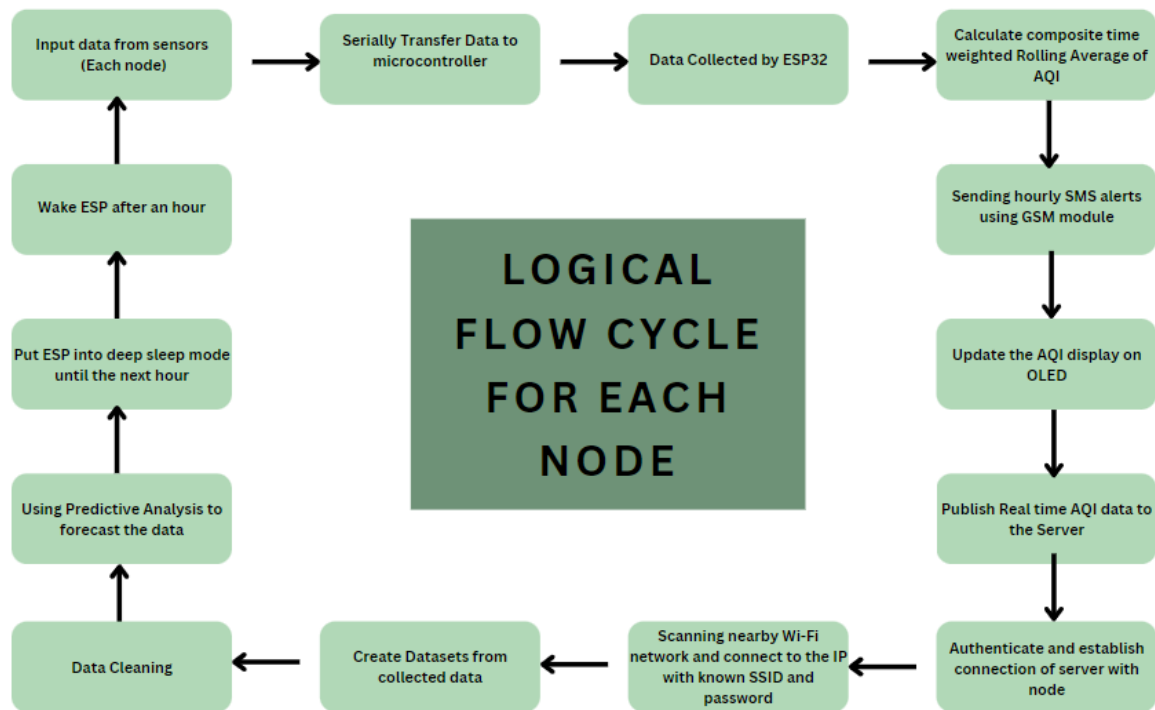


Figure 10: Block Diagram Illustration

4 CHAPTER 3: SYSTEM DETAILS

The individual node is comprised of the necessary sensors connected to the ESP32 microcontroller to measure the concentrations of the air pollutants in the air that are then inserted into the formula for AQI which is then messaged to the user, displayed on node's OLED and uploaded on the cloud server Blynk in real time. The data collected was the stored and further used for Predictive analysis using different machine learning models. This block of the report will go into details about each process mentioned above

4.1 Sensors and their calibration:

The sensors attached to each node included 3 sensors which are connected to the ESP 32:

- i. MQ 7: Measures the concentration of Carbon monoxide
- ii. MQ135: Measures the concentration of Sulfites
- iii. DSM501A: Measures the concentration of PM2.5 and PM1

4.1.1 Calibrating MQ-7:

The internal sensor resistance (RS) is calculated using the formula:

$$RS = \left(\frac{VC - VRL}{VRL} \right) - RL \quad (1)$$

where VC is the circuit voltage, RL is the load resistance, and VRL is the voltage across the load resistor.

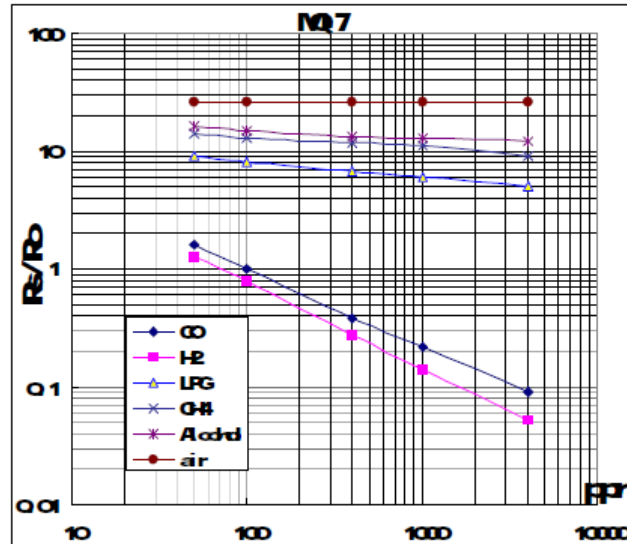


Figure 11: Sensitivity characteristics of MQ7

From the graph, the resistance ratio (RS/RO) in fresh air is observed to be a constant value of 20.8. To find the reference resistance (RO), measure RS in clean air and use the ratio:

$$RS/RO = 20.8 \quad (2)$$

Measure the analog voltage readings from the sensor. Convert these readings to voltage values to determine RS using the formula provided. The graph (Fig. 3.1) shows RS/RO versus ppm for different gases. To calculate gas concentration, read the graph whilst using a log-log scale.

The relationship is linearized using the formula:

$$\log\left(\frac{RS}{RO}\right) = m * \log(x) + b \quad (3)$$

4.1.2 Calibrating MQ-135

The process of calibration for MQ135 follows the same steps as that for MQ7 with the exception of the values from the sensitivity curve. Which is as follows:

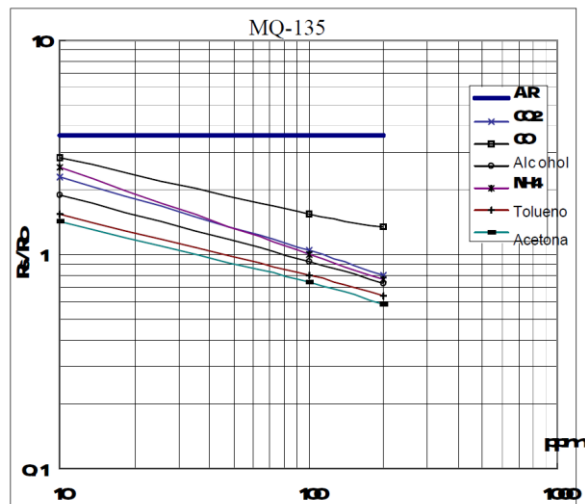


Figure 12: Sensitivity characteristics of MQ135

4.1.3 Calibrating DSM501A

To take readings from the DSM501A sensor firstly we calculated the low ratio by measuring the sum of low pulses received to the GPIO pins of the ESP32 for 30 seconds. The value was then inserted into the equation provided to us in the sensor's datasheet which is as follows:

$$Low\ ratio[\%] = \frac{t(sec)}{30(sec)} * 100 \quad (1)$$

The calculated low ratio is then mapped across the sensor characteristic vs low ratio graph provided in the datasheet which is shown below:

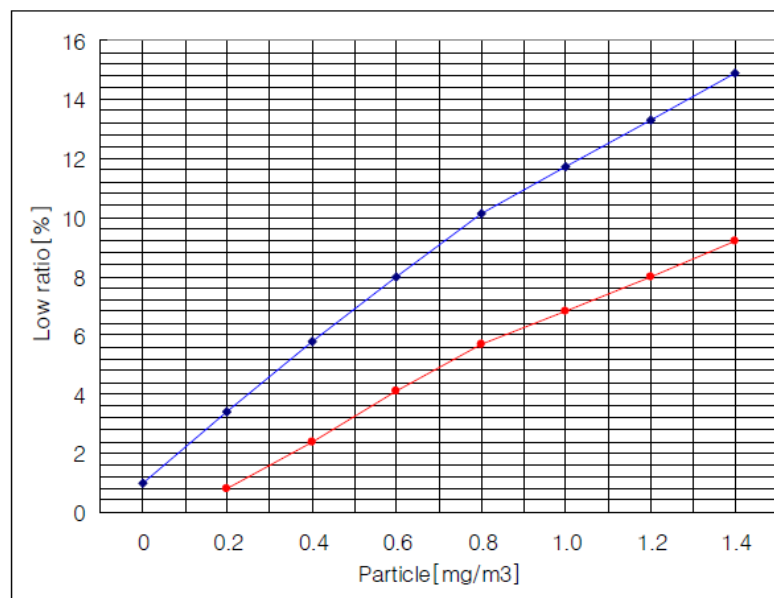


Figure 13: Sensor characteristic vs Low ratio

Using the calculated low ratio percentage and the graphs we deduced the values of PM2.5 and PM1 in mg/m3 dividing each concentration by 1000 we converted our units to ug/m3

4.2 Calculating time weighted AQI

The pollutants used to calculate the AQI included CO, SO2, PM2.5 and PM1 Each pollutant has specific concentration ranges, known as breakpoints, associated with corresponding AQI value ranges that indicate different levels of health concern. To calculate

the AQI, the concentration of each pollutant is measured and its corresponding breakpoint range is identified. A linear equation is then used to calculate the AQI for each pollutant:

$$AQI = (IH - IL) / (CH - CL) \times (C - CL) + IL \quad (1)$$

where IH and IL are the AQI values at the upper and lower ends of the breakpoint range, and CH and CL are the concentration breakpoints at the upper and lower ends of the range, with C being the measured concentration. The breakpoint values are set using the standard EPA'S breakpoint table for each pollutant. [2]

O ₃ (ppm)	PM ₁₀ (ug/m ³)	PM _{2.5} (ug/m ³)	CO (ppm)	SO ₂ (ppm)	NO ₂ (ppm)	AQI Values	Level of Health Concern
0.000 – 0.059	0 – 54	0.0 – 15.4	0.0 – 4.4	0.000 – 0.034	–	0 – 50	Good
0.060 – 0.075	55 – 154	15.5 – 40.4	4.5 – 9.4	0.035 – 0.144	–	51 – 100	Moderate
0.076 – 0.095	155 – 254	40.5 – 65.4	9.5 – 12.4	0.145 – 0.224	–	101 – 150	Unhealthy for Sensitive Groups
0.096 – 0.115	255 – 354	65.5 – 150.4	12.5 – 15.4	0.225 – 0.304	–	151 – 200	Unhealthy
0.116 – 0.374	355 – 424	150.5 – 250.4	15.5 – 30.4	0.305 – 0.604	0.65 – 1.24	201 – 300	Very Unhealthy
–	425 – 504	250.5 – 350.4	30.5 – 40.4	0.605 – 0.804	1.25 – 1.64	301 – 400	Hazardous
–	505 – 604	350.5 – 500.4	40.5 – 50.4	0.805 – 1.004	1.65 – 2.04	401 – 500	Hazardous

Figure 14: EPA's breakpoint and AQI Index [2]

In the process of calculating the composite time-weighted rolling average for AQI, weights are assigned to data points of each pollutant based on their age, with recent data points receiving higher weights through an exponential decay factor of 0.5. Subsequently, the time-weighted rolling average for each pollutant is computed by multiplying each AQI data

point by its respective time-based weight, summing these products, and then dividing by the sum of the weights. Following this, composite weights are determined for each pollutant based on their health impact and regulatory significance, representing their relative importance in the overall air quality index. These composite weights are then used to calculate the weighted rolling average for each pollutant. Finally, the weighted rolling averages of all pollutants are summed to obtain the composite time-weighted rolling average AQI, providing a unified measure that integrates the effects of all included pollutants and reflects the overall air quality.[\[1\]](#)

4.3 Using deep sleep mode of ESP32:

When our node wasn't sampling pollutant values, calculating AQI, or sending data to the cloud server, it would enter sleep mode to conserve energy. This energy-saving feature is crucial for extending the battery life of the device, especially in remote or off-grid locations.

Power mode	Active	Modem-sleep	Light-sleep	Deep-sleep	Hibernation
Sleep pattern	Association sleep pattern			ULP sensor-monitored pattern	-
CPU	ON	ON	PAUSE	OFF	OFF
Wi-Fi/BT baseband and radio	ON	OFF	OFF	OFF	OFF
RTC memory and RTC peripherals	ON	ON	ON	ON	OFF
ULP co-processor	ON	ON	ON	ON/OFF	OFF

Figure 15: Sleep Mode comparison chart [\[3\]](#)

The Real-Time Clock (RTC) of the microcontroller played a vital role by triggering the ESP32 from sleep mode to active mode at scheduled intervals. Upon waking, the ESP32 would sample sensor values for various pollutants and store them in a circular queue. Circular queues were chosen because they efficiently manage a fixed-size buffer, making them ideal

for continuously updating datasets like pollutant levels over time. A total of five circular queues were created: one for SO₂ values, one for CO, one for PM_{2.5}, one for PM₁, and one for AQI. These circular queues were stored in the RTC memory of the ESP32, which is non-volatile, ensuring that all past 24-hour values were retained even when the controller was in sleep mode. This setup allows for continuous monitoring and accurate calculation of AQI by maintaining a rolling history of pollutant concentrations, ensuring data integrity and reliability.

4.4 OLED display

An OLED display is attached to the node's casing to show the values of PM₁ and PM_{2.5} in $\mu\text{g}/\text{m}^3$, SO₂ and CO in ppm, and the final calculated AQI every hour when the ESP32 is triggered from sleep to active mode. This real-time display allows users to quickly and easily monitor air quality data. We evaluated two display options: the SSD1306 and the TFT 128x160. The TFT 128x160 offered a larger and more vibrant display, enhancing readability and providing more space for detailed information. However, it consumed significantly more energy, which was a drawback for our battery-operated nodes designed for extended field deployment. As a result, we opted for the SSD1306 for most of our nodes due to its lower power consumption, compact size, and sufficient resolution for displaying all necessary information clearly. This choice effectively balanced the need for energy efficiency with the requirement for a functional and user-friendly interface.



Figure 16: TFT display



Figure 17: SSD1306 displaying individual pollutant concentrations and the Final AQI

4.5 Sending SMS alerts to users

Using the GSM SIM800L module, we sent hourly updates to our users about air pollutant concentrations. These updates included the values of PM1 and PM2.5 in $\mu\text{g}/\text{m}^3$, SO_2 and CO in ppm, and the calculated AQI. The ESP32 would wake from sleep mode every hour to collect the latest data and transmit it in real-time, ensuring users received up-to-date

information about air quality at the specific geolocation where the node was placed. This system provided users with critical information to help them make informed decisions about their environment. The SIM800L module was chosen for its reliable GSM connectivity and low power consumption, making it suitable for our remote monitoring needs. Only selected phone numbers that were pre-programmed into the code would receive these notifications, ensuring that the updates were sent exclusively to authorized users. This selective notification system enhanced security and ensured that the information reached the right recipients without unnecessary data transmission.

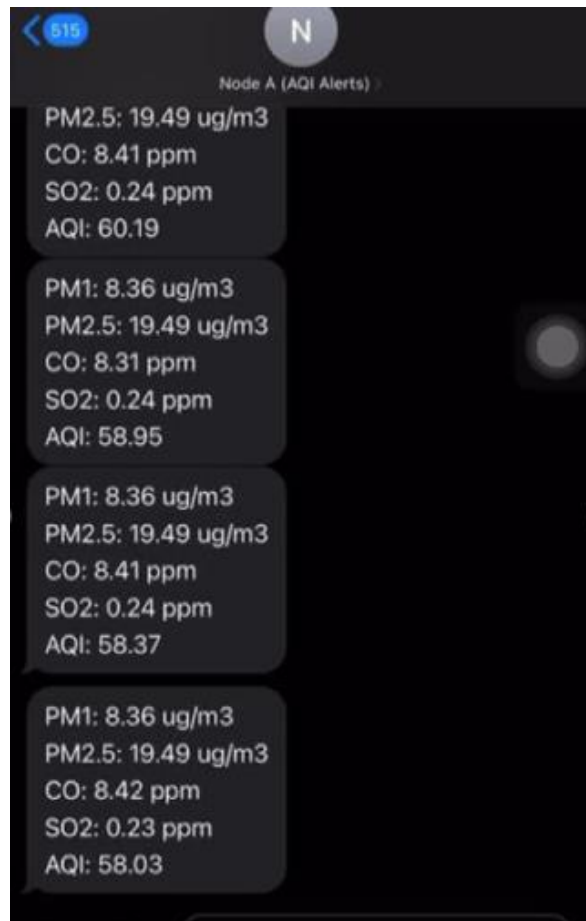


Figure 18: SMS Alerts received on user's phone

4.6 Final circuit designing:

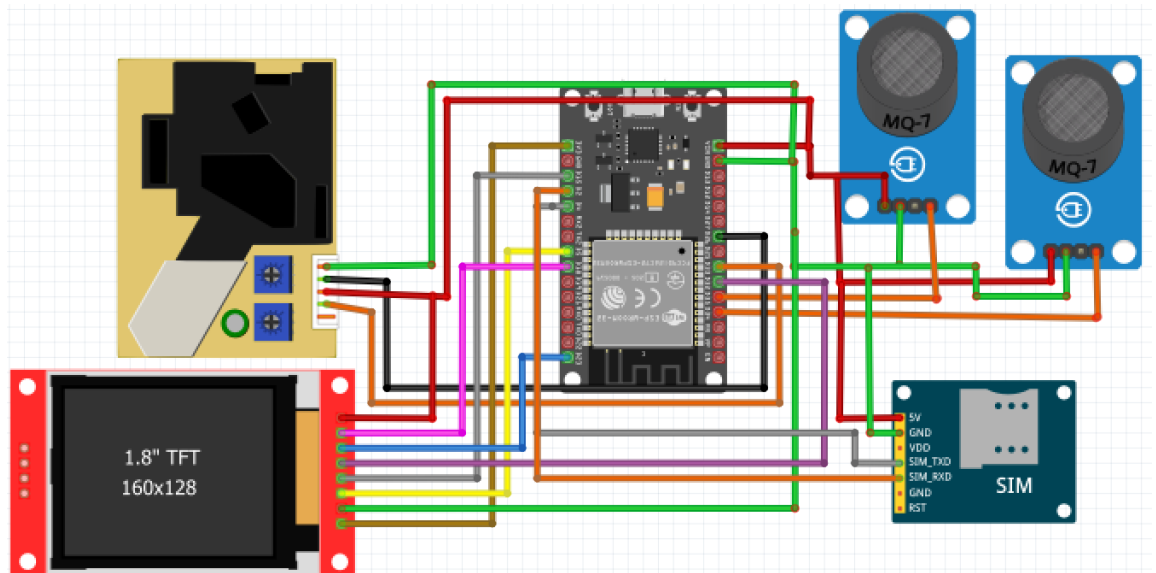


Figure 19: Node A Circuit Schematic

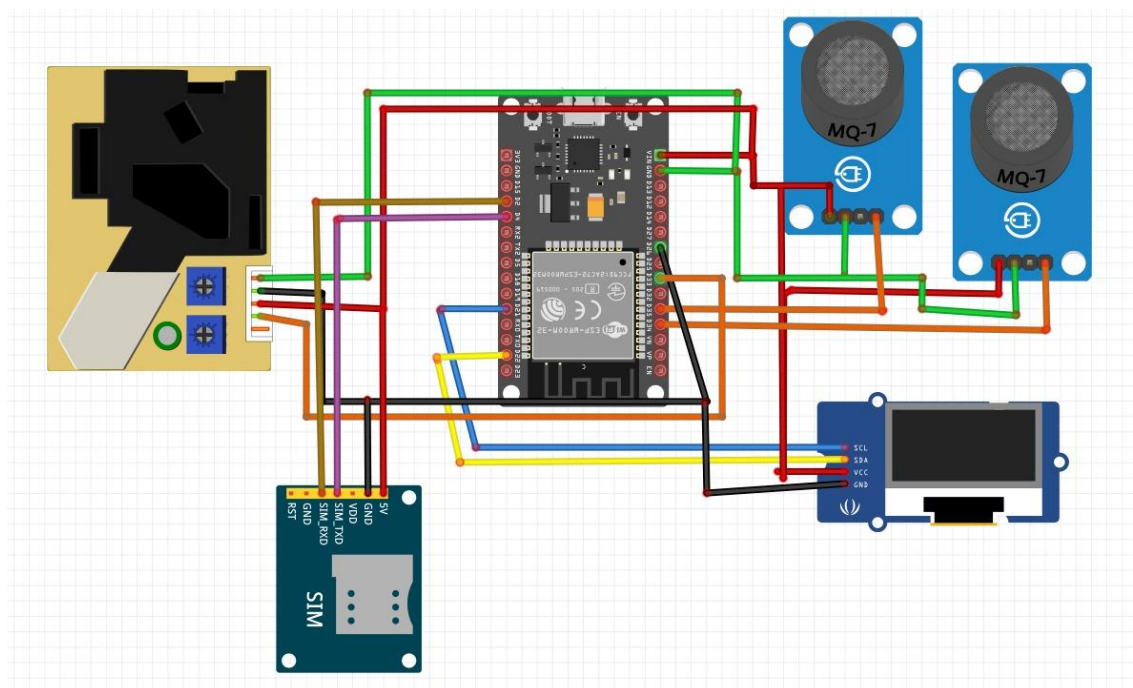


Figure 20: Node B Circuit Schematic

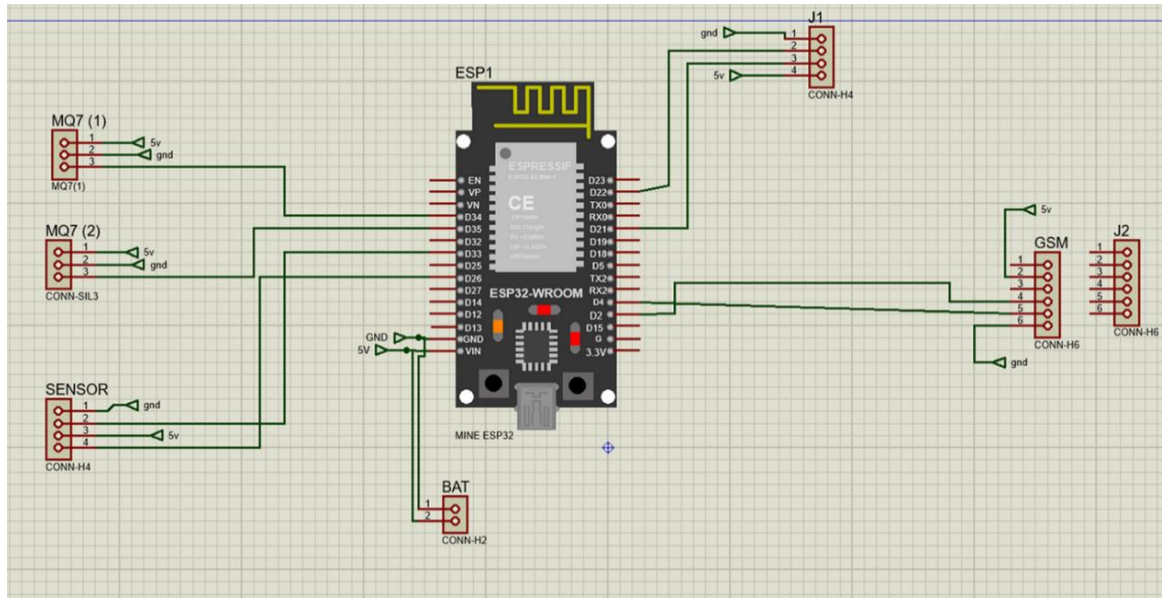
The above circuit of the node was made in the fritzing software. The circuit includes several components connected to the ESP32, each serving a specific function. An MQ-7

sensor is used for measuring carbon monoxide (CO) levels, and an MQ-135 sensor is used for measuring sulfur dioxide (SO₂) levels. A DSM501A sensor is included to measure particulate matter levels (PM_{2.5} and PM₁). The SIM800L GSM module is integrated to send hourly air quality updates via SMS to pre-programmed phone numbers. Additionally, a 1.8" TFT 160x128 display is used for real-time visualization of pollutant concentrations and AQI.

The wiring diagram shows the connections between these components and the ESP32. The red wires represent connections to the 5V supply of the ESP32, while the green wires represent ground connections.

4.7 PCB designing:

Firstly, the design of our air quality monitoring node's schematic was designed using Proteus software. schematic diagram detailed the connections between the ESP32 microcontroller, MQ-7 sensors for CO detection, MQ-135 sensor for SO₂ detection, DSM501A sensor for PM_{2.5} and PM₁ detection, and the SIM800L GSM module for SMS notifications. Female headers were used to place the sensors, OLED and sim module on top of them to form stable connections with the ESP32 board.



Secondly, the PCB layout was generated in Proteus, highlighting the routing connections to ensure no interference between unwanted connections. The PCB design composes the components onto a compact board, facilitating easy assembly and deployment of the monitoring node.

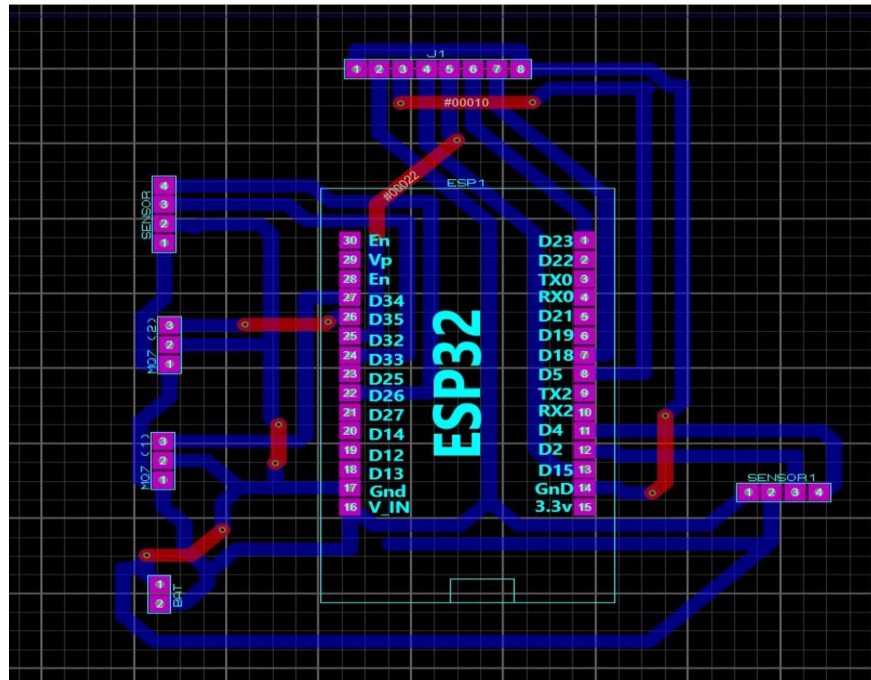


Figure 22: PCB design of node A

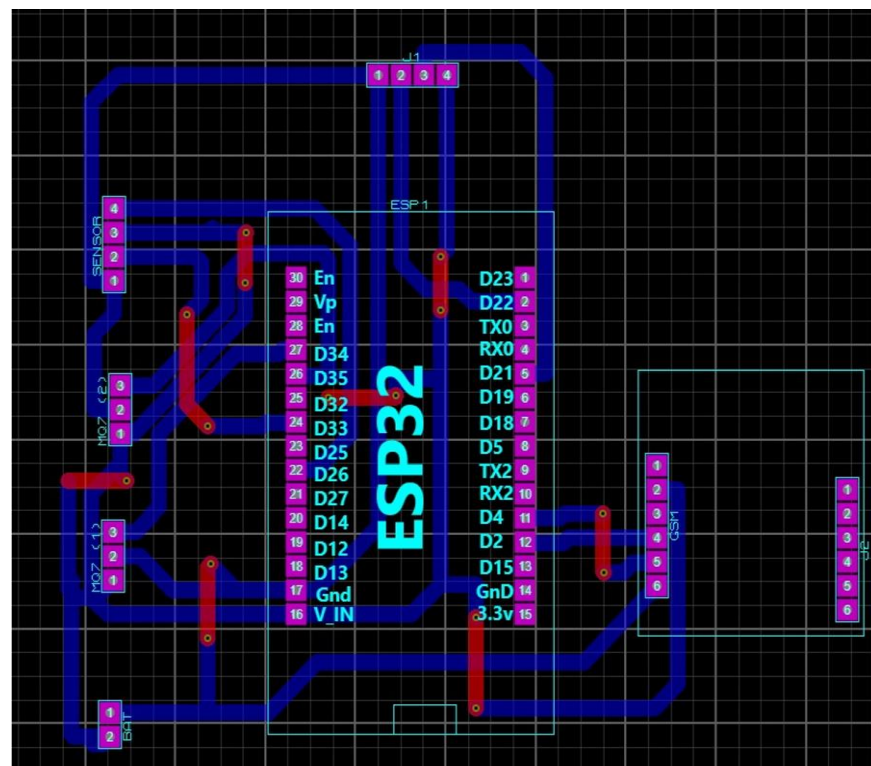


Figure 23: PCB Design of Node B

4.8 CAD 3D Casing designing:

We further designed a protective casing using Fusion 360 software. This custom casing was meticulously crafted to house the circuit and sensors securely, ensuring durability and protection from external elements. Specific slots were created within the casing to accommodate the sensors and the OLED display, allowing them to function optimally while remaining protected. After finalizing the design, we fabricated the casing and installed the circuit and sensors into their respective slots. This integration ensured that the air quality monitoring node was ready for deployment in various environmental conditions.

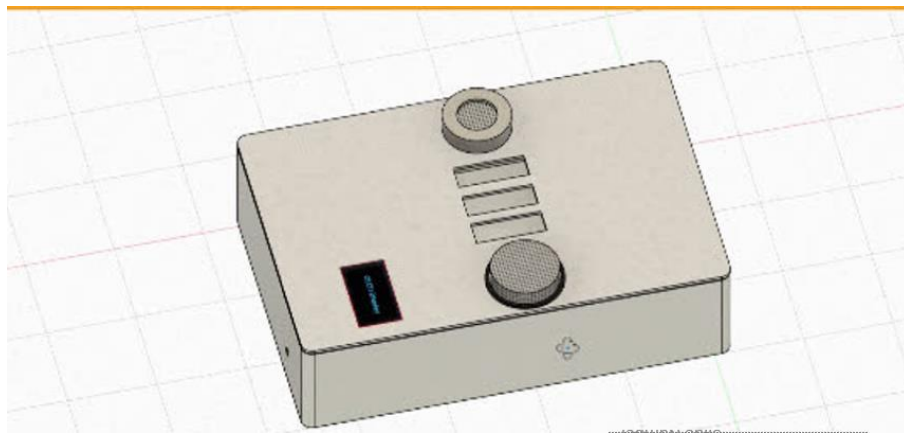


Figure 24: 3D Model of Node Casing



Figure 25: Side View and dimensions of Node's casing



Figure 26: Top View of Node Casing

4.9 Cloud Setup

As this project revolves around cloud connected AQI network, each node is created to publish individual pollutant concentrations and final AQI to a cloud platform using the Message Queuing Telemetry Transfer (MQTT) protocol.

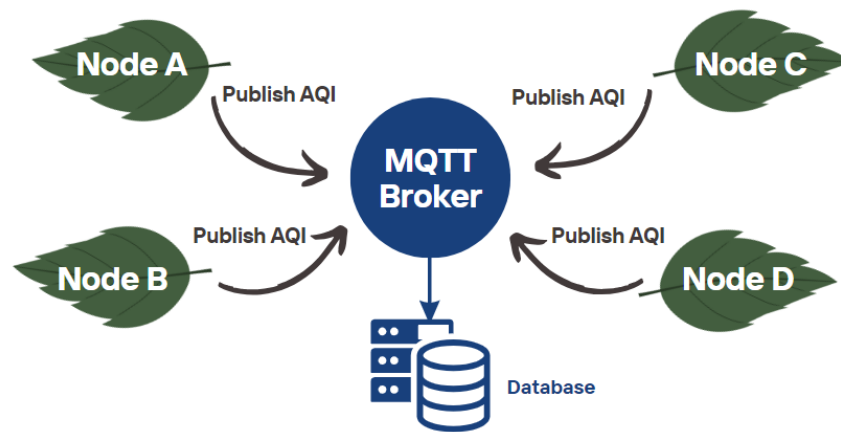


Figure 27: Cloud Network Structure

The cloud platform used for data collection was AWS IoT Core, while Blynk IoT Cloud was employed for data visualization.

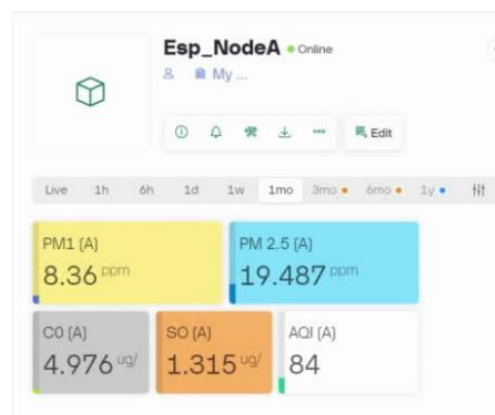


Figure 28: Data Visualization on Cloud

The data reception process begins with the ESP32 connecting to a specified Wi-Fi network using the pre-defined SSID and password. Essential cloud authentications, including certificates, Template ID, Device ID, and Authorization Token, are used to securely connect to the project's cloud platform. The MQTT protocol's topic definition establishes a known path for the nodes and central broker to publish data in JSON format. Once published, the data is stored in the cloud database. The dataset, obtained in CSV format, is then used for further data processing.

4.10 Dataset:

We placed our node at a specific geolocation in Islamabad and followed the described process to collect hourly values of SO₂, CO, PM₁, PM_{2.5} and AQI. This data collection took place from March 10 to June 30, resulting in approximately 2,717 values for each parameter, recorded at one-hour intervals. During this period, the ESP32 would wake from sleep mode every hour to sample the sensors, calculate the AQI, store the data, send updates via the GSM SIM800L module and upload data to cloud server in real time. This approach provided us with a dataset for analyzing air quality trends over nearly 3 months, allowing us to gather insights into pollution patterns in Islamabad.

	A	B	C	D	E	F	G
1	Datetime	PM_2.5	PM_1	SO2	CO	Total_AQI	Category
2	10/03/2024 18:00	14.101	21.523	1.12	9.44	298	Poor
3	10/03/2024 19:00	14.323	24.347	1.19	10.54	298	Poor
4	10/03/2024 20:00	14.895	26.144	1.18	12.71	300	Poor
5	10/03/2024 21:00	14.845	26.06	1.03	13.65	302	Very Poor
6	10/03/2024 22:00	12.438	22.754	0.88	13.47	302	Very Poor
7	10/03/2024 23:00	10.593	20.406	0.75	12.82	301	Very Poor
8	11/03/2024 0:00	9.034	17.613	0.72	12.08	301	Very Poor
9	11/03/2024 1:00	7.748	15.754	0.72	10.94	301	Very Poor
10	11/03/2024 2:00	7.188	14.461	0.72	9.32	302	Very Poor
11	11/03/2024 3:00	7.154	15.389	0.91	8.56	302	Very Poor
12	11/03/2024 4:00	7.856	17.296	1.42	8.89	302	Very Poor
13	11/03/2024 5:00	10.686	25.154	2.08	10.22	302	Very Poor
14	11/03/2024 6:00	14.146	30.153	2.5	11.76	302	Very Poor
15	11/03/2024 7:00	16.826	32.762	2.86	13.24	302	Very Poor
16	11/03/2024 8:00	19.412	35.889	3.35	14.17	303	Very Poor
17	11/03/2024 9:00	21.652	37.894	3.38	15.01	304	Very Poor
18	11/03/2024 10:00	23.008	37.346	3.21	15.36	305	Very Poor
19	11/03/2024 11:00	22.759	36.223	2.85	14.38	307	Very Poor
20	11/03/2024 12:00	20.951	32.38	2.2	13.87	308	Very Poor
21	11/03/2024 13:00	18.924	27.779	1.56	12.09	309	Very Poor
22	11/03/2024 14:00	16.084	23.1	1.35	12.13	309	Very Poor
23	11/03/2024 15:00	13.554	19.735	1.15	11.34	309	Very Poor
24	11/03/2024 16:00	12.327	18.454	1.07	10.72	308	Very Poor
25	11/03/2024 17:00	11.878	18.448	1.13	11.72	308	Very Poor
26	11/03/2024 18:00	11.806	19.778	1.39	12.08	307	Very Poor
27	11/03/2024 19:00	13.148	23.955	1.53	11.87	306	Very Poor
28	11/03/2024 20:00	14.806	27.077	1.43	13.52	306	Very Poor
29	11/03/2024 21:00	17.187	30.047	1.32	14.21	307	Very Poor
30	11/03/2024 22:00	17.130	30.776	1.3	15.01	300	Very Poor

Figure 29: Collected Dataset in CSV format

4.11 Process for predictive analysis:

After collecting our dataset, we took 9 steps to perform predictive analysis on our data.

1. Data visualization
2. Parameter correlation with AQI
3. Data splitting
4. Training Linear Regression model
5. Training Random Forest model

6. Training Gradient Boosting model
7. Training CatBoost Regressor model
8. Model evaluation test comparison

4.11.1 Data Visualization:

After collecting the relevant dataset, we performed data visualization by importing the data from an Excel file and displaying it in various visual formats. Graphs and charts were created to represent the trends and patterns of the air pollutant concentrations and AQI over time, making it easier to understand the data at a glance.

4.11.2 Parameter correlation with AQI:

We then conducted a correlation analysis to determine the relationship between the different parameters (SO₂, CO, PM1, and PM2.5) and the AQI. By calculating correlation coefficients and displaying them in a heatmap, we confirmed positive correlations between the pollutant concentrations and AQI, indicating that as pollutant levels increased, so did the AQI.

4.11.3 Training Linear Regression model:

For predictive analysis, we first used linear regression, a straightforward method that models the relationship between a dependent variable (AQI) and one or more independent variables (pollutant concentrations). This model provided a baseline for understanding how each parameter individually impacted AQI.

4.11.4 Training Random Forest model:

Next, we applied the Random Forest algorithm, which is an ensemble learning method that constructs multiple decision trees during training and outputs the mean prediction of

the individual trees. This method is effective for capturing complex interactions between parameters and improving prediction accuracy.

4.11.5 Training Gradient boosting model:

We also employed Gradient Boosting, which builds an ensemble of trees sequentially, where each tree attempts to correct the errors of the previous one. This approach focuses on minimizing prediction errors and enhancing the model's performance through iterative improvement.

4.11.6 Training CatRegressor model:

CatBoost Regressor, a gradient boosting algorithm developed specifically for categorical features, was also utilized. This model is known for its fast-training process and high prediction accuracy, making it suitable for handling the structured dataset of air quality parameters and AQI.

4.11.7 Model Evaluation Tests:

We later used a series of tests, as outlined in the table, to compare the performance of the predictive models. The tests included:

4.11.7.1 Training Time:

This measures the duration it takes for each model to learn from the training data. It is crucial for evaluating the efficiency of the models, especially when dealing with large datasets

4.11.7.2 Training Score:

This reflects how well the model fits the training data, providing an initial indication of the model's capability.

4.11.7.3 Test Score:

This evaluates the model's performance on the testing data, helping to assess its generalizability and predictive power.

4.11.7.4 R2 Score:

This metric indicates how well the predicted values match the actual values, with higher scores representing better performance.

4.11.7.5 RMSE (Root Mean Square):

This measures the average magnitude of the errors between predicted and actual values, with lower values indicating higher accuracy

4.11.7.6 MAPE (Mean Absolute Percentage Error):

This calculates the percentage error between predicted and actual values, providing a normalized measure of prediction accuracy.

These tests enabled us to compare the models comprehensively, identifying the most accurate and efficient model for predicting air quality based on the collected data.

5 CHAPTER 4: CODE EXPLANTIONS

5.1 C programming on ESP32

ESP 32 was programmed using the Arduino IDE in C language to interface with the sensors, GSM module, OLED and establishing Cloud connecting using its Wi-Fi module. The code contains various in built and user defined function for structured execution and improved readability. All functions are called in void setup() and user defined functions, GPIO pins, Wi-Fi authentication credentials, necessary library headers and module definitions are declared outside of it.

5.1.1 Wi-Fi declarations and credentials

This section of the code sets up and configures the Blynk platform and WiFi for an ESP32-based air quality monitor. It defines the Blynk credentials (template ID, device name, auth token, and template name) and includes necessary libraries (`<WiFi.h>` and `<BlynkSimpleEsp32.h>`). Wi-Fi credentials (auth token, SSID, and password) are stored in arrays. A `BlynkTimer` object is initialized for scheduling tasks, and the `temp` function is declared for temperature-related operations.

```
#define BLYNK_TEMPLATE_ID "TMPL67dUlm2V-"
#define BLYNK_DEVICE_NAME "AQI Monitor 4"
#define BLYNK_AUTH_TOKEN "GfOC21XlbaQC3fkpHGIilV6VDJYspK57"
#define BLYNK_TEMPLATE_NAME "AQI Monitor 4"

#define BLYNK_PRINT Serial
#include <WiFi.h>
#include <BlynkSimpleEsp32.h>

char auth[] = "GfOC21XlbaQC3fkpHGIilV6VDJYspK57";
char ssid[] = "Ames"; // type your wifi name
char pass[] = "amnaahmad"; // type your wifi password

BlynkTimer timer;
void temp();
```

Figure 30: Wi-Fi Declarations

5.1.2 Global Variables

These Global variables are defined to store values which are to be sent to the cloud and have to be used globally throughout the cloud.

```
//global variables
float pm1_d; //V0
float pm25_d; //V1
float co_d; //V2
float so_d; //V3
float aqi_d; //V4
//=====================================================
```

Figure 31: Global variables for Cloud data transfer and storage

5.1.3 OLED declarations

This code sets up the OLED display for an ESP32 project. It includes the necessary libraries and defines the display's width (128 pixels) and height (64 pixels). The reset pin is defined as `OLED\_RESET` with a value of -1, indicating it is not used, and the I2C address is set to `0x3C`. An `Adafruit\_SSD1306` object named `display` is created with these parameters.

```
//OLED declarations
#include <SPI.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#define SCREEN_WIDTH 128 // OLED display width, in pixels
#define SCREEN_HEIGHT 64 // OLED display height, in pixels

// Declaration for SSD1306 display connected using I2C
#define OLED_RESET -1 // Reset pin
#define SCREEN_ADDRESS 0x3C
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);
```

Figure 32: OLED declarations

5.1.4 Sleep Mode Declarations

This code configures the ESP32 for deep sleep mode. It defines a conversion factor (`uS_TO_S_FACTOR`) to convert microseconds to seconds and sets the sleep duration (`TIME_TO_SLEEP`) to 3 seconds. The variables `bootCount` and `runcount` are declared with the `RTC_DATA_ATTR` attribute, allowing them to retain their values during deep sleep cycles.

```
..
//for ESP deep sleep mode
#define uS_TO_S_FACTOR 1000000 /* Conversion factor for micro seconds to seconds */
#define TIME_TO_SLEEP  3      /* Time ESP32 will go to sleep (in seconds) */

RTC_DATA_ATTR int bootCount = 0;
RTC_DATA_ATTR int runcount = 0;
```

Figure 33: Sleep Mode declarations

5.1.5 GPIO pin definitions and necessary variables' initialization

This part of the code declares various variables and pin definitions. The `#define` statements assign pin numbers for PM1, PM2.5, CO, and SO sensors. The `unsigned long` and `float` variables are used for timing, sensor data, and calculations related to air quality indices (AQI). The `composite weightage` variables hold weights for different pollutants in the AQI calculation.

```
#define PM1PIN 26
#define PM25PIN 33
#define CPIN 35
#define SPIN 34
unsigned long duration1;
unsigned long duration2;
unsigned long starttime;
unsigned long sampletime_ms = 3000; //sampe 10s ;
unsigned long lowpulseoccupancy1 = 0;
unsigned long lowpulseoccupancy2 = 0;
```

Figure 34: GPIO pins and variables declaration (part 1)

```

float ratio1 = 0;
float ratio2 = 0;
float concentration1 = 0;
float concentration2 = 0;
float tempSO;
float tempCO;
float composite_weightage_PM25=0.50;
float composite_weightage_PM1=0.25;
float composite_weightage_CO=0.15;
float composite_weightage_SO=0.10;
float rc;
float AQI25;
float AQI1;
float AQICO;
float AQISO;
float hourly_aqi;

```

Figure 35: GPIO pins and variable declaration (part 2)

5.1.6 Individual pollutant concentration functions' declaration

This part of the code deals with the declaration of all individual pollutant concentrations' calculator functions.

```

float calculateCOPPM();
float calculateSO2PPM();
void PM1andPM2calc();
float calculatePM25_AQI();

```

Figure 36: Individual Pollutant concentration functions' declaration

5.1.7 RTC data array for 24 values

This part of the code defines circular buffers for storing air quality data over time for PM1, PM2.5, CO, and SO concentrations. Each buffer can hold up to 'maxSize' (24) readings which are the hourly values throughout the day. The 'RTC_DATA_ATTR' attribute ensures these variables retain their values during deep sleep mode of the ESP32. Arrays 'pm1Arr', 'pm25Arr', 'coArr', and 'soArr' store the readings, while 'pm1Front', 'pm1Rear', 'pm25Front', 'pm25Rear', 'coFront', 'coRear', 'soFront', and 'soRear' keep track of the front and rear indices for each buffer, initialized to -1 indicating they are empty.

```
const int maxSize = 24;

RTC_DATA_ATTR float pm1Arr[maxSize];
RTC_DATA_ATTR int pm1Front = -1;
RTC_DATA_ATTR int pm1Rear = -1;

RTC_DATA_ATTR float pm25Arr[24];
RTC_DATA_ATTR int pm25Front = -1;
RTC_DATA_ATTR int pm25Rear = -1;

RTC_DATA_ATTR float coArr[maxSize];
RTC_DATA_ATTR int coFront = -1;
RTC_DATA_ATTR int coRear = -1;

RTC_DATA_ATTR float soArr[maxSize];
RTC_DATA_ATTR int soFront = -1;
RTC_DATA_ATTR int soRear = -1;
```

Figure 37: RTC data array for 24 values

5.1.8 Circular Queue Functions

The 'Queue_enqueue' function manages the insertion of values into a circular queue implemented using an array 'arr'. When the queue is empty, it initializes both 'front' and

`rear` to 0 and inserts the `value` at `rear`. If the queue is not full, it increments `rear` to the next position in the circular array and inserts the `value` there. When the queue is full, it replaces the oldest value by incrementing `front` to skip over the oldest element and adjusts `rear` to insert the `value` in the next available slot, effectively implementing a circular buffer. This approach ensures efficient handling of both empty and full states of the queue while maintaining the integrity of its circular structure.

```
bool Queue_is_Empty(int front, int rear) {
    return front == -1 && rear == -1;
}

bool Queue_is_Full(int front, int rear) {
    return (rear + 1) % maxSize == front;
}

void Queue_enqueue(float value, float arr[], int& front, int& rear) {
    if (Queue_is_Empty(front, rear)) {
        front = rear = 0;
        arr[rear] = value;
    } else if (!Queue_is_Full(front, rear)) {
        rear = (rear + 1) % maxSize;
        arr[rear] = value;
    } else {
        // Queue is full, replace the oldest value
        front = (front + 1) % maxSize;
        rear = (rear + 1) % maxSize;
        arr[rear] = value;
    }
}
```

Figure 38: Queuing Function definition

The `Queue\_display` function checks if the circular queue, stored in array `arr` between `front` and `rear`, is empty using "Queue_is_Empty". If empty, it prints "Queue is empty!". Otherwise, it initializes `i` from `front` and iterates through the queue with a do-while loop, printing each element at `arr[i]`. Index `i` wraps around using modulo arithmetic `(i + 1) % maxSize` until it reaches `(rear + 1) % maxSize`, ensuring all elements are

displayed. This function efficiently displays the circular queue's contents on Arduino's serial monitor.

```
void Queue_display(float arr[], int front, int rear) {
    if (Queue_is_Empty(front, rear)) {
        Serial.println("Queue is empty!");
    } else {
        int i = front;

        do {
            Serial.print(arr[i]);
            Serial.print(" ");
            i = (i + 1) % maxSize;
        } while (i != (rear + 1) % maxSize);

        Serial.println();
    }
}
```

Figure 39: Queuing display function

5.1.9 OLED Display Function

The 'displayReadings' function updates an OLED display with PM1, PM2.5, CO, SO2, and AQI readings. It starts by clearing the display and setting text size to small with white color. Each parameter (pm1, pm2, co, so2, aqi) is displayed with its respective unit (ug/m3 for PM1 and PM2.5, ppm for CO and SO2, and no unit for AQI). After printing all values, the display is updated, and a 5-second delay is added to maintain visibility. This function efficiently presents air quality data on the OLED screen.

```
//=====OLED DISPLAY FUNCTION=====

void displayReadings(float pm1, float pm2, float co, float so2, float aqi) {
    display.clearDisplay();

    display.setTextSize(1);
    display.setTextColor(WHITE);

    display.setCursor(0, 0);
    display.print("PM1: ");
    display.print(pm1);
    display.println(" ug/m3");

    display.setCursor(0, 10);
    display.print("PM2.5: ");
    display.print(pm2);
    display.println(" ug/m3");

    display.setCursor(0, 20);
    display.print("CO: ");
    display.print(co);
    display.println(" ppm");

    display.setCursor(0, 30);
    display.print("SO2: ");
    display.print(so2);
    display.println(" ppm");

    display.setCursor(0, 40);
    display.print("AQI: ");
    display.print(aqi);

    display.display();
    delay(5000);
}

```

Figure 40: OLED display function definition

5.1.10 Function for sending sensor data to cloud

The ‘sendSensor’ function sends sensor data to Blynk Cloud using virtual pins: PM1 (‘pm1_d’) to V0, PM2.5 (‘pm25_d’) to V1, CO (‘co_d’) to V2, SO2 (‘so_d’) to V3, and AQI (‘aqi_d’) to V4, each followed by a 2-second delay between subsequent transmissions. It logs each transmission with a message to the Serial monitor confirming data sent to Blynk Cloud, ensuring continuous updates for real-time air quality monitoring and analysis.

```

..
void sendSensor()
{
    Blynk.virtualWrite(V0, pm1_d);
    Blynk.virtualWrite(V1, pm25_d);
    Blynk.virtualWrite(V2, co_d);
    delay(2000);
    Blynk.virtualWrite(V3, so_d);
    Blynk.virtualWrite(V4, aqi_d);
    Serial.println("Sending Data to Blynk Cloud");
}

```

Figure 41: Function for sending data to Cloud

5.1.11 VoidSetup ()

The ‘setup’ function initializes the ESP microcontroller and sets up necessary configurations for operation. It begins by initializing serial communication at a baud rate of 115200. If the OLED display fails to initialize with the SSD1306 driver, an error message is printed, and the program halts. Following successful initialization, it clears the display buffer. Depending on the ‘bootCount’, it either executes initialization routines or enters a regular operation mode. During initialization (‘bootCount == 0’), it calculates PM1 (‘concentration1’) and PM2.5 (‘concentration2’) values, CO (‘tempCO’), and SO2 (‘tempSO’). It displays these values on the OLED screen using ‘displayReadings’ and enqueues them into respective circular queues (‘pm1Arr’, ‘pm25Arr’, ‘coArr’, ‘soArr’) while printing the enqueued values to Serial. Upon subsequent runs (‘bootCount > 0’), it repeats these operations, increments ‘runcount’, and calculates hourly AQI based on historical data in the circular queues. It updates the OLED display with the calculated AQI and prepares data (‘pm1_d’, ‘pm25_d’, ‘co_d’, ‘so_d’, ‘aqi_d’) for transmission to Blynk Cloud. Finally, it sets the ESP to sleep mode, waking up periodically for measurements and data transmission.


```

void setup() {
  Serial.begin(115200);
  //=====
  if(!display.begin(SSD1306_SWITCHCAPVCC, SCREEN_ADDRESS)) {
    Serial.println(F("SSD1306 allocation failed"));
    for(;;); // Don't proceed, loop forever
  }

  // Clear the buffer.
  display.clearDisplay();

  if(bootCount == 0) //Run this only the first time
  {
    bootCount = bootCount+1;
    Serial.print("boot count number : ");
    Serial.println(bootCount);

    runcount = runcount +1;

    PM1andPM2calc();

    float pm1=concentration1;//variable1
    float pm2=concentration2;//variable2
    tempCO = calculateCOPPM();//variable3
    tempSO = calculateSO2PPM(); //variable4

    //OLED DISPLAY PART
    displayReadings(pm1, pm2, tempCO, tempSO, 0.0); // Initially, AQI is 0
  }
}

```

Figure 42:Void Setup (part 1)

```

Queue_enqueue(concentration1, pm1Arr, pm1Front, pm1Rear);
Serial.print("Enqueued PM1: ");
Serial.println(concentration1);
Queue_display(pm1Arr, pm1Front, pm1Rear);

Queue_enqueue(concentration2, pm25Arr, pm25Front, pm25Rear);
Serial.print("Enqueued PM2.5: ");
Serial.println(concentration2);
Queue_display(pm25Arr, pm25Front, pm25Rear);

```

Figure 43: Void Setup (part 2)

```

Queue_enqueue(tempCO, coArr, coFront, coRear);
Serial.print("Enqueued CO: ");
Serial.println(tempCO,4);
Queue_display(coArr, coFront, coRear);

Queue_enqueue(tempSO, soArr, soFront, soRear);
Serial.print("Enqueued SO2: ");
Serial.println(tempSO,4);
Queue_display(soArr, soFront, soRear);

Serial.println("-----");

}else

```

Figure 44: Void Setup (part 3)

```

, ----
{
    PM1andPM2calc();
    float pm1=concentration1;/**
    float pm2=concentration2;/**
    tempCO = calculateCOPPM(); /**
    tempSO = calculateSO2PPM();/**

    //OLED DISPLAY PART
    displayReadings(pm1, pm2, tempCO, tempSO, 0.0); // Initially, AQI is 0

    Queue_enqueue(concentration1, pm1Arr, pm1Front, pm1Rear);
    Serial.print("Enqueued PM1: ");
    Serial.println(concentration1);
    Queue_display(pm1Arr, pm1Front, pm1Rear);

    Queue_enqueue(concentration2, pm25Arr, pm25Front, pm25Rear);
    Serial.print("Enqueued PM2.5: ");
    Serial.println(concentration2);
    Queue_display(pm25Arr, pm25Front, pm25Rear);

```

Figure 45: Void Setup (part 4)

```

Queue_enqueue(tempCO, coArr, coFront, coRear);
Serial.print("Enqueued CO: ");
Serial.println(tempCO,4);
Queue_display(coArr, coFront, coRear);

Queue_enqueue(tempSO, soArr, soFront, soRear);
Serial.print("Enqueued SO2: ");
Serial.println(tempSO,4);
Queue_display(soArr, soFront, soRear);

Serial.println("-----");

```

Figure 46: Void Setup (part 5)

```

    runcount = runcount +1;
    Serial.print("runcount number : ");
    Serial.println(runcount);
}

delay(3000);

if (runcount>=2){
    rc=runcount;
    AQI25 = calculatePM25_AQI(pm25Arr, maxSize, pm1Front, pm1Rear);
    Serial.println(AQI25,4);

    AQI1 = calculatePM1_AQI(pm1Arr, maxSize, pm25Front, pm25Rear);
    Serial.println(AQI25,4);

    AQISO = calculateSO_AQI(soArr, maxSize, soFront, soRear);
    Serial.println(AQISO,4);

    AQICO = calculateCO_AQI(coArr, maxSize, coFront, coRear);
    Serial.println(AQICO,4);

    hourly_aqi= (AQI25*0.35)+(AQI1*0.25)+(AQISO*0.2)+(AQICO*0.2) ;

    Serial.print("hourly aqi is : ");
    Serial.println(hourly_aqi);
    // Update OLED display with AQI
    displayReadings(concentration1, concentration2, tempCO, tempSO, hourly_aqi);
}

```

Figure 47: Void Setup (part 6)

```

    pm1_d=concentration1; //V0
    pm25_d=concentration2; //V1
    co_d=tempCO; //V2
    so_d=tempSO; //V3
    aqi_d=hourly_aqi; //V4

    //sending data to cloud platform
    Blynk.begin(auth, ssid, pass);
    for(int i=0;i<2;i++){
        delay(3000);
        timer.setInterval(1000L, sendSensor);
        delay(3000);
        temp();
    }
}

//=====

    display.clearDisplay();
    display.display();

    Serial.println("ESP GOING TO SLEEP");

    esp_sleep_enable_timer_wakeup(35 * uS_TO_S_FACTOR);

    esp_deep_sleep_start();

}

```

Figure 48: Void Setup (part 7)

5.1.12 Blynk.run function for delay

This function is defined to provide a certain execution time for the ‘Blynk.run’ and ‘timer.run’ commands. This function when called in setup provides enough delay time for complete transfer of data to Blynk’s Cloud platform.

```

void temp() {
    Blynk.run();
    timer.run();
}

```

Figure 49: Execution time function for Blynk.run() command

5.1.13 PM1 and PM2.5 data function

The `PM1andPM2calc` function measures durations of low pulses from PM1 and PM2.5 sensors, accumulates these in `lowpulseoccupancy1` and `lowpulseoccupancy2`, respectively, and calculates occupancy ratios over a 30-second sample period. Using predefined conversion curves from sensor specifications, it computes concentrations for PM1 (`concentration1`) and PM2.5 (`concentration2`). The function loops until the sample time ends, ensuring continuous monitoring and calculation of particulate concentrations.

```

//pm 1 and pm 2.5 data function

void PM1andPM2calc() {
    duration1 = pulseIn(PM1PIN, LOW);
    duration2 = pulseIn(PM25PIN, LOW);
    lowpulseoccupancy1 = lowpulseoccupancy1+duration1;
    lowpulseoccupancy2 = lowpulseoccupancy2+duration2;

    pointx:
    if ((millis()-starttime) > sampletime_ms)//if the sampel time == 30s
    {

        ratio1 = ((lowpulseoccupancy1/1000.0)/(30.0))*100; // Integer percentage 0=>100
        //y=mx+b
        //y=(9.67*x)+1.66
        //x=(b-y)/m
        concentration1 = (ratio1-1.666)/9.67; // using spec sheet curve
        ratio2 = ((lowpulseoccupancy2/1000.0)/(30.0))*100; // Integer percentage 0=>100
        //y=7.25*x-0.7
        concentration2 = (ratio2+0.7)/7.25; //
        starttime = millis();
    }
    else{
        goto pointx;
    }
}

```

Figure 50: PM1 and PM2.5 data function

5.1.14 CO sensor ppm function

The 'calculateCOPPM' function reads analog values from a CO sensor (MQ-7) through multiple samples, computes the average, and converts these values into voltage. It then calculates the sensor resistance (RS_gas) based on the voltage, compares it to a reference resistance (R0), and derives a ratio. Using predefined slope (m) and y-intercept (b) values specific to the sensor's calibration curve, it converts the ratio into a logarithmic scale representing CO concentration in parts per million (ppm). The function returns this ppm value, providing a measure of CO concentration detected by the sensor.

```
//CO sensor function
float calculateCOPPM() {
    float temp_sensorValue = 0;
    float m = -1.6706; // Slope
    float b = 1.347;   // Y-Intercept
    float R0 = 7.116;  // Sens
    float sensor_volt; // Define variable for sensor voltage
    float RS_gas;      // Define variable for sensor resistance
    float ratio;       // Define variable for ratio
    int sensorValue;   // Variable to store the analog values from MQ-7
    float ppm_log; // Get ppm value in linear scale according to the ratio value
    float ppm;      // Convert ppm value to log scale
    for (int i = 0; i < 50; i++) {
        temp_sensorValue = temp_sensorValue + analogRead(CPIN); // Read analog values of sensor
    }
    sensorValue = temp_sensorValue / 50;
    sensor_volt = sensorValue * (5.0 / 4096.0); // Convert analog values to voltage
    RS_gas = ((5.0-sensor_volt) / sensor_volt) * 10000; // Get value of RS in a gas
    ratio = RS_gas / (R0);
    ppm_log = (log10(ratio) - b) / m;
    ppm = pow(10, ppm_log); // Convert ppm value to log scale
    return ppm;
}
/**
```

Figure 51: CO concentration (ppm) function

5.1.15 SO2 sensor ppm Function

The ``calculateSO2PPM`` function reads analog values from an SO2 sensor (MQ-135) over several samples to compute an average. It converts these values into voltage and calculates the sensor resistance (RS_gas). By comparing this resistance to a reference resistance (R0), it determines a ratio representing the concentration of SO2 in the air. Using predefined slope (m) and y-intercept (b) values specific to the sensor's calibration curve, it converts this ratio into a logarithmic scale representing SO2 concentration in parts per million (ppm).

```
//SO sensor function
float calculateSO2PPM() {
    float m = -0.4376;           // Slope
    float b = 0.7165;            // Y-Intercept
    float R0 = 0.92;             // Sensor Resistance in fresh air
    float sensor_volt;           // Define variable for sensor voltage
    float RS_gas;                // Define variable for sensor resistance
    float ratio;                 // Define variable for ratio
    int sensorValue;             // Variable to store the analog values from MQ-135
    float ppm_log;               // Get ppm value in linear scale according to the ratio value
    float ppm;                   // Convert ppm value to log scale
    float temp_sensorValue = 0;

    for (int i = 0; i < 5000; i++) {
        temp_sensorValue = temp_sensorValue + analogRead(SPIN); // Read analog values of sensor
    }

    sensorValue = temp_sensorValue / 5000;
    sensor_volt = sensorValue * (50.0 / 4096.0);

    // Convert analog values to voltage
    //RS_gas = ((5.0 * 1.0) / sensor_volt) - 1.0; // Get value of RS in a gas
    ratio = sensor_volt / R0; // Get ratio RS_gas/RS_air
    ppm_log = (log10(ratio) - b) / m; // Get ppm value in linear scale according to the ratio value
    ppm = abs((pow(10, ppm_log)*10)-(0.19*rc)); // Convert ppm value to log scale

    return ppm;
}
```

Figure 52: SO2 concentration ppm function

5.1.16 Air Quality standard mapping conditions

The ``printAirQualityDescription`` function interprets the Air Quality Index (AQI) to provide concise descriptions of current air conditions. It categorizes AQI values into several

levels: "good" (0-50), "moderate" (51-100), "unhealthy for sensitive groups" (101-150), "unhealthy" (151-200), "very unhealthy" (201-300), and "hazardous" (300+). Each category corresponds to escalating levels of health concern, guiding users on the potential impacts of air quality on their well-being based on recognized standards.

```
//=====
// Function to inform the user about the type of air based on AQI values
void printAirQualityDescription(float aqi) {
    if (aqi <= 50) {
        Serial.println("Air quality is good.");
    } else if (aqi <= 100) {
        Serial.println("Air quality is moderate.");
    } else if (aqi <= 150) {
        Serial.println("Air quality is unhealthy for sensitive groups.");
    } else if (aqi <= 200) {
        Serial.println("Air quality is unhealthy.");
    } else if (aqi <= 300) {
        Serial.println("Air quality is very unhealthy.");
    } else {
        Serial.println("Air quality is hazardous.");
    }
}
```

Figure 53: AQI mapping function

5.1.17 AQI calculator function for PM2.5

The ‘calculatePM25_AQI’ function computes the Air Quality Index (AQI) for PM2.5 concentrations using data stored in the array ‘arr’. It calculates the PM2.5 concentration (‘pm25Concentration’) through a weighted average and assigns an AQI based on predefined breakpoints. The function categorizes PM2.5 levels into six AQI ranges and computes the AQI using a linear formula derived from these breakpoints. This numerical assessment is essential for monitoring PM2.5 pollution levels and understanding their potential health impacts.


```

//calculating aqi for pm2.5
float calculatePM25_AQI(float arr[], int maxsize, int Front, int Rear) {
    float pm25Concentration;
    pm25Concentration = weighted_average( arr, maxsize, composite_weightage_PM25, Front, Rear);
    float aqi;
    // AQI=((I_hi-I_lo)/(BP_hi-BP_lo))*(Cp-BP_lo)+I_lo; //AQI formula
    if (pm25Concentration <= 12.0) {
        aqi = (50.0 / 12) * pm25Concentration;
    } else if (pm25Concentration <= 35.4) {
        aqi = 51.0 + ((100.0 - 50.0) / (35.4 - 12.1)) * (pm25Concentration - 12.1);
    } else if (pm25Concentration <= 55.4) {
        aqi = 101.0 + ((150.0 - 100.0) / (55.4 - 35.5)) * (pm25Concentration - 35.5);
    } else if (pm25Concentration <= 150.4) {
        aqi = 151.0 + ((200.0 - 150.0) / (150.4 - 55.5)) * (pm25Concentration - 55.5);
    } else if (pm25Concentration <= 250.4) {
        aqi = 201.0 + ((300.0 - 200.0) / (250.4 - 150.5)) * (pm25Concentration - 150.5);
    } else {
        aqi = 301.0 + ((500.0 - 300.0) / (350.4 - 250.5)) * (pm25Concentration - 250.5);
    }
    Serial.print("pm25 aqi = ");
    Serial.println(aqi);

    return aqi;
}

```

Figure 54: PM 2.5's AQI calculator function

5.1.18 AQI Calculator Function for PM1

The 'calculatePM1_AQI' function calculates the Air Quality Index (AQI) for PM1 concentrations using data stored in the array 'arr'. It computes the PM1 concentration ('pm1Concentration') by taking a weighted average from the array. The function then determines the appropriate AQI category based on predefined breakpoints for PM1 levels. It assigns an AQI value using a linear interpolation formula derived from these breakpoints, ensuring that the AQI reflects the severity of PM1 pollution accurately

```
//=====
//calculating aqi for pml
float calculatePM1_AQI(float arr[], int maxsize, int Front, int Rear) {
    float pmlConcentration;
    pmlConcentration = weighted_average( arr, maxsize, composite_weightage_PM1, Front, Rear);
    float aqi;
    // AQI=((I_hi-I_lo)/(BP_hi-BP_lo))*(Cp-BP_lo)+I_lo;//AQI formula
    if (pmlConcentration <= 12.0) {
        aqi = (50.0 / 12) * pmlConcentration;
    } else if (pmlConcentration <= 35.4) {
        aqi = 51.0 + ((100.0 - 50.0) / (35.4 - 12.1)) * (pmlConcentration - 12.1);
    } else if (pmlConcentration <= 55.4) {
        aqi = 101.0 + ((150.0 - 100.0) / (55.4 - 35.5)) * (pmlConcentration - 35.5);
    } else if (pmlConcentration <= 150.4) {
        aqi = 151.0 + ((200.0 - 150.0) / (150.4 - 55.5)) * (pmlConcentration - 55.5);
    } else if (pmlConcentration <= 250.4) {
        aqi = 201.0 + ((300.0 - 200.0) / (250.4 - 150.5)) * (pmlConcentration - 150.5);
    } else {
        aqi = 301.0 + ((500.0 - 300.0) / (350.4 - 250.5)) * (pmlConcentration - 250.5);
    }
    Serial.print("pml aqi = ");
    Serial.println(aqi);
    return aqi;
}
```

Figure 55: PM1's AQI calculator function

5.1.19 AQI Calculator function for CO

The `calculateCO\_AQI` function calculates the Air Quality Index (AQI) for carbon monoxide (CO) based on a weighted average from the `arr` array. It evaluates the CO concentration (`coConcentration`) and assigns an AQI value using predefined thresholds. These thresholds categorize CO levels into various AQI ranges using linear interpolation, providing a precise indication of CO pollution severity.

```

//calculating aqi for CO
float calculateCO_AQI(float arr[], int maxsize, int Front, int Rear) {
    float coConcentration;
    coConcentration = weighted_average( arr, maxsize, composite_weightage_CO, Front, Rear);
    float aqi;
    // AQI=((I_hi-I_lo)/(BP_hi-BP_lo))*(Cp-BP_lo)+I_lo;//AQI formula
    if (coConcentration <= 4.4) {
        aqi = (50.0 / 4.4) * coConcentration;
    } else if (coConcentration <= 9.4) {
        aqi = 51.0 + ((100.0 - 50.0) / (9.4 - 4.4)) * (coConcentration - 4.4);
    } else if (coConcentration <= 12.4) {
        aqi = 101.0 + ((150.0 - 100.0) / (12.4 - 9.4)) * (coConcentration - 9.4);
    } else if (coConcentration <= 15.4) {
        aqi = 151.0 + ((200.0 - 150.0) / (15.4 - 12.4)) * (coConcentration - 12.4);
    } else if (coConcentration <= 30.4) {
        aqi = 201.0 + ((300.0 - 200.0) / (30.4 - 15.4)) * (coConcentration - 15.4);
    } else {
        aqi = 301.0 + ((500.0 - 300.0) / (50.4 - 30.4)) * (coConcentration - 30.4);
    }

    Serial.print("co aqi = ");
    Serial.println(aqi);
    return aqi;
}

```

Figure 56: CO's AQI calculator function

5.1.20 AQI calculator for SO2

The ‘calculateSO_AQI’ function determines the Air Quality Index (AQI) for sulfur dioxide (SO₂) by averaging concentrations from ‘arr’. It applies breakpoints to categorize SO₂ levels into AQI ranges via linear interpolation, producing an ‘aqi’ value that assesses pollution levels and informs measures to address air quality concerns. This function plays a crucial role in evaluating SO₂ contamination and guiding strategies to mitigate its environmental and health effects.

```

//calculating aqi for SO2
float calculateSO_AQI(float arr[], int maxsize, int Front, int Rear) {
    float sulfiteConcentration;
    sulfiteConcentration = weighted_average( arr, maxsize, composite_weightage_SO, Front, Rear);
    float aqi;
    // AQI=((I_hi-I_lo)/(BP_hi-BP_lo))*(Cp-BP_lo)+I_lo;//AQI formula

    if (sulfiteConcentration <= 0.035) {
        aqi = (50.0 / 0.035) * sulfiteConcentration;
    } else if (sulfiteConcentration <= 0.144) {
        aqi = 50.0 + ((100.0 - 50.0) / (0.144 - 0.035)) * (sulfiteConcentration - 0.035);
    } else if (sulfiteConcentration <= 0.304) {
        aqi = 100.0 + ((150.0 - 100.0) / (0.304 - 0.144)) * (sulfiteConcentration - 0.144);
    } else if (sulfiteConcentration <= 0.604) {
        aqi = 150.0 + ((200.0 - 150.0) / (0.604 - 0.304)) * (sulfiteConcentration - 0.304);
    } else if (sulfiteConcentration <= 1.004) {
        aqi = 200.0 + ((300.0 - 200.0) / (1.004 - 0.604)) * (sulfiteConcentration - 0.604);
    } else {
        aqi = 300.0 + ((500.0 - 300.0) / (1.504 - 1.004)) * (sulfiteConcentration - 1.004);
    }
    Serial.print("SO2 aqi = ");
    Serial.println(aqi);

    return aqi;
}

```

Figure 57: SO2 's AQI calculator function

5.1.21 Weighted Average Calculator Function

The ‘weighted_average’ function calculates an average where recent data has more impact than older data. It uses a technique where each data point is multiplied by a decreasing power of 2, meaning newer data points contribute more to the average than older ones. This helps in reflecting current trends or conditions based on the sequence of data received. The function computes the weighted sum of values and divides it by the sum of the weights to produce a weighted average result, which is useful for applications like monitoring changes over time.

```

..
//calculating weighted average*
float weighted_average(float values[], int maxsize, float composite_weightage, int Front, int Rear){
    float sum = 0.0;
    float divider = 1.0;
    float result;
    int first_value;
    float t;
    double temp_composite_weightage=0;
    // int size = sizeof(values) / sizeof(values[0]);

    for (int i = Front; i <= Rear; i++) {
        first_value= Rear-i;
        sum = sum + (values[first_value]* pow(2,-i));
        Serial.print(values[first_value], 5);
        Serial.print("+");
        temp_composite_weightage= pow(2,-i) ;
        Serial.print(temp_composite_weightage, 18);
        Serial.print("=");
        Serial.println(sum, 4);
        if (values[i] > 0) {
            divider = divider + pow(2, -i);
        }
    }

    if (runcount>25){
        for (int i = 0; i < 24; i++) {
            first_value= 23-i;
            sum = sum + (values[first_value]* pow(2,-i));
            Serial.print(values[first_value], 5);
            Serial.print("+");
            temp_composite_weightage= pow(2,-i) ;
            Serial.print(temp_composite_weightage, 18);
            Serial.print("=");

```

Figure 58: Weighted Average function (part 1)

```

        Serial.println(sum, 4);
        if (values[i] > 0) {
            divider = divider + pow(2, -i);
        }
    }

    result = sum / divider;
    Serial.print(sum, 4);
    Serial.print("/");
    Serial.print(divider, 4);
    Serial.print("=");
    Serial.println(result, 4);

    return result;
}

```

Figure 59: Weighted Average function (part 2)

5.2 Predictive analysis:

5.2.1 Data visualization:

Our first step was to read dataset from excel file and view it in our chosen environment which was python jupyter notebook

```

[3]: # Display the first few rows of the DataFrame
print(node1_data.head())
# displaying the end of the dataset
print(node1_data.tail())

```

Figure 60: Data visualization code (part 1)

The code reads the dataset into a DataFrame using the pandas library and then prints the first and last few rows to give a snapshot of the data.

```
[4]: #displaying the datatypes for column of dataset
node1_data.info()
```

Figure 61: Data Visualization code (part 2)

The code is then used to display the structure and data types of the columns in the dataset. By using the `info()` method on the DataFrame, we can obtain a concise summary that includes the number of non-null entries in each column, the data type of each column, and the memory usage of the DataFrame.

```
[5]: # Plotting the data
plt.figure(figsize=(10, 9))
date_time=node1_data['Datetime']
total_aqi=node1_data['Total_AQI']
plt.plot(node1_data['Datetime'], node1_data['Total_AQI'], marker='o')
plt.title('NODE1 AQI VS TIME')
plt.xlabel('TIME')
plt.ylabel('Total Aqi')
plt.grid(True)
plt.show()
```

Figure 62: Data visualization code (part 3)

We then visualize the Air Quality Index (AQI) data over time for Node1. By plotting the Datetime against Total_AQI, we can observe how AQI values change over the recorded period.

5.2.2 Data Correlation:

```
: selected_columns = ['Total_AQI', 'PM_2.5', 'PM_1', 'SO2', 'CO']
correlation = node1_data[selected_columns].corr(method='kendall')
print(correlation)
corr= node1_data[selected_columns].corr()
ax= sns.heatmap(corr, vmin=-1, vmax=1, center=0, cmap=sns.diverging_palette(10,220,n=220), square=True, annot=True)
ax.set_xticklabels(ax.get_xticklabels(), rotation=45, horizontalalignment='right');
```

Figure 63: Data correlation code

The purpose of this code is to analyze and visualize the correlations between various air quality parameters and the Total AQI. This helps in understanding the relationships

between different pollutants and their combined effect on the air quality. The Kendall correlation method is used to compute the correlation matrix, which measures the strength and direction of the association between pairs of variables.

5.2.3 Data Splitting;

```
#dividing data into attributes and labels
X= pd.DataFrame(np.c_[node1_data['PM_2.5'],node1_data['PM_1'],node1_data['SO2'],node1_data['CO']],columns = ['PM_2.5','PM_1','SO2','CO'])
Y=node1_data['Total_AQI']
train_index = int(0.8*len(X))
#splitting x and y data into test and train data
X_train, X_test = X[:train_index], X[train_index:]
Y_train, Y_test = Y[:train_index], Y[train_index:]

# Print the shapes of the datasets
print(f"X_train shape: {X_train.shape}")
print(f"X_test shape: {X_test.shape}")
print(f"Y_train shape: {Y_train.shape}")
print(f"Y_test shape: {Y_test.shape}")
```

Figure 64: Code to split dataset

This code is to prepare the dataset for training and testing machine learning models. It involves dividing the data into attributes (features) and labels (target), and then splitting this data into training and testing sets. The attributes are defined by creating a DataFrame X containing the columns PM_2.5, PM_1, SO2, and CO. The labels (target) are defined as the Total_AQI column from the dataset, forming YResults explanation.

5.2.4 Linear Regression:

```
# Standardizing the data
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Initialize the LinearRegression model
regressor = LinearRegression()

# Start the timer
start0 = time.process_time()

# Fit the model
regressor.fit(X_train, Y_train)

# End the timer
end0 = time.process_time()

# Calculate and print the training time
training_time = end0 - start0
print("Training time:", training_time, "s")

# Print the intercept and coefficients
print('Intercept:', regressor.intercept_)
print('Coefficients:', regressor.coef_)
```

Figure 65: Linear Regression code

First, it standardizes the training and testing data using StandardScaler to ensure that all features have a mean of zero and a standard deviation of one. The LinearRegression model is then initialized and trained on the standardized training data.

5.2.5 Random Forest model:

```
# Applying Random Forest (method 2)
rf= RandomForestRegressor(n_estimators=100,min_samples_split= 6,min_samples_leaf= 1, max_features= 'log2',max_depth= 100, bootstrap= True)
start0= time.process_time()
rf.fit(X_train, Y_train)
stop0= time.process_time()
training_time =stop0-start0
print("Training time:", training_time, "s")
start1 = time.process_time()
Y_predicted = rf.predict(X_test)
stop1= time.process_time()
# Calculate and print the prediction time
predict_time = stop1 - start1
print("Predict time:", predict_time, "s")
rf_train_score=rf.score(X_train, Y_train)
rf_test_score=rf.score(X_test, Y_test)
print("Random forest training score", rf_train_score)
print("Random forest testing score", rf_test_score)

# Additional metrics for evaluation
mse = mean_squared_error(Y_test, Y_predicted)
r2 = metrics.r2_score(Y_test, Y_predicted)
rms = np.sqrt(metrics.mean_squared_error(Y_test, Y_predicted))
mape = mean_absolute_percentage_error(Y_test, Y_predicted)

print(f'Mean Squared Error: {mse}')
print(f'R2 Score: {r2}')
print(f'root mean square error: {rms}')
print(f'Mean Absolute Percentage Error: {mape}')
```

Figure 66: Random Forest Model code

The regressor (rf) is initialized with 100 estimators (n_estimators), a minimum of 6 samples required to split an internal node (min_samples_split), 1 sample required to be at a leaf node (min_samples_leaf), 'log2' as the maximum number of features considered for splitting (max_features), a maximum depth of 100 for each tree (max_depth), and uses bootstrapped samples (bootstrap=True). It measures training and prediction times using time.process_time() to evaluate computational efficiency. After training on X_train and Y_train, it predicts outcomes for X_test and computes training and testing scores (rf_train_score, rf_test_score) to assess model performance. This approach ensures that the Random Forest model is effectively tuned and evaluated based on its configured hyperparameters and computational performance.

5.2.6 Gradient Boost Regressor:

```
#Applying GradientBoostRegressor (method 3)

gbm= GradientBoostingRegressor(n_estimators=50, max_depth=10, learning_rate=0.1, max_features='sqrt',min_samples_split=2,min_samples_leaf=2)
start0= time.process_time()
gbm.fit(X_train, Y_train)
stop0=time.process_time()
training_time=stop0-start0
print('training time', training_time)
start1= time.process_time()
Y_predicted= gbm.predict(X_test)
stop1= time.process_time()
gbm_train_score=gbm.score(X_train, Y_train)
gbm_test_score=gbm.score(X_test,Y_test)

print("GBM training score:", gbm_train_score)
print("GBM test score:", gbm_test_score)

# Additional metrics for evaluation
mse = mean_squared_error(Y_test, Y_predicted)
r2 = metrics.r2_score(Y_test, Y_predicted)
rms = np.sqrt(metrics.mean_squared_error(Y_test, Y_predicted))
mape = mean_absolute_percentage_error(Y_test, Y_predicted)

print(f'Mean Squared Error: {mse}')
print(f'R2 Score: {r2}')
print(f'root mean square error: {rms}')
print(f'mean absolute percentage error: {mape}')

# Calculate histogram data
bins = 10
actual_counts, bin_edges = np.histogram(Y_test, bins=bins)
predicted_counts, _ = np.histogram(Y_predicted, bins=bin_edges)
percentage_diff = (predicted_counts - actual_counts) / actual_counts * 100
# Plot histograms
plt.hist(Y_test, bins=bin_edges, alpha=0.5, label='Actual Values', color='blue', edgecolor='black')
plt.hist(Y_predicted, bins=bin_edges, alpha=0.5, label='Predicted Values', color='red', edgecolor='black')
# Annotate percentage differences
for i in range(len(bin_edges) - 1):
```

Figure 67: Gradient Boost Regressor code

The above code snippet initializes a GradientBoostingRegressor using Python's scikit-learn library with the following hyperparameters: 50 estimators (n_estimators), maximum depth of 10 for each tree (max_depth), learning rate of 0.1 (learning_rate), 'sqrt' as the maximum number of features considered for splitting (max_features), minimum of 2 samples required to split an internal node (min_samples_split), and 2 samples required to be at a leaf node (min_samples_leaf).

It then measures the training time using time.process_time() before and after fitting the model (gbm.fit(X_train, Y_train)), providing insights into computational efficiency. After training, it predicts outcomes for X_test using gbm.predict(X_test) and calculates

training and testing scores (gbm_train_score, gbm_test_score) using gbm.score() to evaluate model performance on both training and testing datasets (X_train, Y_train, X_test, Y_test).

This method allows for the optimization and evaluation of the Gradient Boosting Regressor based on its specified hyperparameters

5.2.7 CAT Boost Regressor:

```
# Define the parameter grid for CatBoost
param_distributions = {
    'iterations': [100, 150, 200, 600, 500, 800, 1000],
    'learning_rate': [0.01, 0.1, 0.2, 0.3],
    'depth': [4, 6, 7, 8, 9, 10],
    'l2_leaf_reg': [1, 2, 3, 4, 5, 6, 7, 8, 9],
    'border_count': [32, 64, 55, 128, 150, 254]
}

# Create a CatBoostRegressor object
cbr = CatBoostRegressor(random_seed=42, silent=True)

# Create a RandomizedSearchCV object
random_search = RandomizedSearchCV(estimator=cbr, param_distributions=param_distributions,
                                   n_iter=100, cv=5, verbose=2, random_state=42, n_jobs=-1)

# Fit the model to the training data
start0 = time.process_time()
random_search.fit(X_train, Y_train)
stop0 = time.process_time()
training_time = stop0 - start0
print("Training time:", training_time, "s")

# Best parameters
print("Best parameters found by random search:", random_search.best_params_)

# Make predictions on the test set
start1 = time.process_time()
Y_predicted = random_search.predict(X_test)
stop1 = time.process_time()
predict_time = stop1 - start1
print("Predict time:", predict_time, "s")

# Evaluate the model
cbr_train_score = random_search.score(X_train, Y_train)
cbr_test_score = random_search.score(X_test, Y_test)
mse = mean_squared_error(Y_test, Y_predicted)
r2 = r2_score(Y_test, Y_predicted)
rms = np.sqrt(mse)
mape = mean_absolute_percentage_error(Y_test, Y_predicted)
```

Figure 68: CAT Boost Regressor Model code

The code defines a parameter grid (param_distributions) encompassing key hyperparameters such as iterations, learning rate, tree depth, L2 regularization, and border

count. A CatBoostRegressor instance (cbr) is initialized with a fixed random seed and silenced output to facilitate stable and quiet training. Employing RandomizedSearchCV, the code conducts a randomized search cross-validation to explore 100 different combinations (n_iter=100) across 5 folds (cv=5), leveraging parallel computing for efficiency (n_jobs=-1). The model is then trained on the provided training dataset (X_train, Y_train), and the best-performing hyperparameters identified by the search are reported. Subsequently, predictions are generated for the test dataset (X_test), and performance metrics such as training and testing scores (cbr_train_score, cbr_test_score), mean squared error (mse), R-squared (r2), and root mean squared error (rms) are computed to assess model accuracy and robustness.

6 CHAPTER 05: RESULTS

6.1 Arduino Serial Monitor

The first display observed on the serial Monitor is the RTC deep sleep reset and its clock parameters. The numerically queued values are the enqueued sampled value of all four pollutants (PM2.5, PM1, CO, SO2).

```
rst:0x5 (DEEPSLEEP_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
configsip: 0, SPIWP:0xee
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode:DIO, clock div:1
load:0x3fff0030,len:1344
load:0x40078000,len:13964
load:0x40080400,len:3600
entry 0x400805f0
0.0000
Enqueued PM1: 28.65
29.14 29.03 29.56 29.58 29.99 28.18 27.83 29.06 28.76 29.71 29.74 28.93 29.99 28.58 27.59 29.90 28.83 28.85 28.87 27.84 28.08 28.67 27.83 28.65
Enqueued PM2.5: 23.19
22.89 20.39 18.09 16.99 17.69 24.19 18.79 24.49 19.69 19.89 21.99 20.19 21.39 17.99 15.19 17.49 20.49 17.89 20.49 19.69 22.09 21.69 16.29 23.19
Enqueued CO: 17.5858
17.58 17.58 17.53 17.55 17.55 17.54 17.52 17.56 17.55 17.53 17.54 17.51 17.40 17.53 17.58 17.61 17.56 17.61 17.63 17.64 17.62 17.55 17.60 17.59
Enqueued SO2: 0.3189
0.32 0.32 0.32 0.32 0.32 0.32 0.32 0.32 0.32 0.32 0.32 0.32 0.32 0.32 0.32 0.32 0.32 0.32 0.32 0.32 0.32 0.32 0.32 0.32
-----
```

Figure 69: Serial Monitor display (part 1)

```
runcount number : 34
17.98700*1.0000000000000000=17.9870
21.38700*0.5000000000000000=28.6805
20.18700*0.2500000000000000=33.7272
21.98700*0.1250000000000000=36.4756
19.88700*0.0625000000000000=37.7186
19.68700*0.0312500000000000=38.3338
24.48700*0.0156250000000000=38.7164
18.78700*0.0078125000000000=38.8632
24.18700*0.0039062500000000=38.9576
17.68700*0.0019531250000000=38.9922
16.98700*0.0009765625000000=39.0088
18.08700*0.0004882812500000=39.0176
20.38700*0.0002441406250000=39.0226
22.88700*0.0001220703125000=39.0254
23.18700*0.0000610351562500=39.0268
16.28700*0.0000305175781250=39.0273
21.68700*0.0000152587890625=39.0276
22.08700*0.00000762939453125=39.0278
19.68700*0.000003814697265625=39.0279
20.48700*0.000001907348632812=39.0279
17.88700*0.000000953674316406=39.0279
20.48700*0.000000476837158203=39.0279
17.48700*0.000000238418579102=39.0279
15.18700*0.000000119209289551=39.0279
39.0279/3.0000=13.0093
pm25 aqi = 52.95
```

Figure 70: Serial Monitor display (part 2)

```

28.57600*1.0000000000000000=28.5760
29.98600*0.5000000000000000=43.5690
28.92600*0.2500000000000000=50.8005
29.73600*0.1250000000000000=54.5175
29.70600*0.0625000000000000=56.3741
28.75600*0.0312500000000000=57.2727
29.05600*0.0156250000000000=57.7267
27.82600*0.0078125000000000=57.9441
28.17600*0.0039062500000000=58.0542
29.98600*0.0019531250000000=58.1128
29.57600*0.0009765625000000=58.1416
29.55600*0.0004882812500000=58.1561
29.02600*0.0002441406250000=58.1632
29.13600*0.0001220703125000=58.1667
28.64600*0.0000610351562500=58.1685
27.82600*0.0000305175781250=58.1693
28.66600*0.00001525878906250=58.1698
28.07600*0.000007629394531250=58.1700
27.83600*0.000003814697265625=58.1701
28.86600*0.000001907348632812=58.1701
28.84600*0.000000953674316406=58.1702
28.82600*0.000000476837158203=58.1702
29.89600*0.000000238418579102=58.1702
27.58600*0.000000119209289551=58.1702
58.1702/3.0000=19.3901
pm1 aqi = 66.64

```

Figure 71: Serial Monitor display (part 3)

```

0.32058*1.0000000000000000=0.3206
0.32645*0.5000000000000000=0.4838
0.32182*0.2500000000000000=0.5643
0.32058*0.1250000000000000=0.6043
0.32099*0.0625000000000000=0.6244
0.32058*0.0312500000000000=0.6344
0.32016*0.0156250000000000=0.6394
0.32141*0.0078125000000000=0.6419
0.32099*0.0039062500000000=0.6432
0.31975*0.0019531250000000=0.6438
0.32058*0.0009765625000000=0.6441
0.32224*0.0004882812500000=0.6443
0.32016*0.0002441406250000=0.6444
0.31975*0.0001220703125000=0.6444
0.31892*0.0000610351562500=0.6444
0.31975*0.0000305175781250=0.6444
0.31975*0.00001525878906250=0.6444
0.31851*0.000007629394531250=0.6444
0.31810*0.000003814697265625=0.6444
0.31851*0.000001907348632812=0.6444
0.31892*0.000000953674316406=0.6444
0.31934*0.000000476837158203=0.6444
0.31975*0.000000238418579102=0.6444
0.31934*0.000000119209289551=0.6444
0.6444/3.0000=0.2148
SO2 aqi = 122.13

```

Figure 72: Serial Monitor display (part 4)

```

17.53347*1.0000000000000000=17.5335
17.39802*0.5000000000000000=26.2325
17.51258*0.2500000000000000=30.6106
17.54393*0.1250000000000000=32.8036
17.53347*0.0625000000000000=33.8995
17.55439*0.0312500000000000=34.4480
17.56485*0.0156250000000000=34.7225
17.52303*0.0078125000000000=34.8594
17.54393*0.0039062500000000=34.9279
17.55439*0.0019531250000000=34.9622
17.55439*0.0009765625000000=34.9793
17.53347*0.0004882812500000=34.9879
17.57532*0.0002441406250000=34.9922
17.57532*0.0001220703125000=34.9943
17.58578*0.0000610351562500=34.9954
17.59626*0.0000305175781250=34.9960
17.55439*0.0000152587890625=34.9962
17.61723*0.00000762939453125=34.9964
17.63822*0.000003814697265625=34.9964
17.62772*0.000001907348632812=34.9965
17.60675*0.000000953674316406=34.9965
17.56485*0.000000476837158203=34.9965
17.60675*0.000000238418579102=34.9965
17.57532*0.000000119209289551=34.9965
34.9965/3.0000=11.6655
co aqi = 138.76

```

Figure 73: Serial Monitor display (part 5)

The Final AQI is displayed on the serial Monitor after the display of the individual AQIs of each pollutant. ESP32 then connects with the Wi-Fi connection with the SSID and Password defined in the code which is validated when the IP address of that Wi-Fi connection is displayed on the serial Monitor. Following this, ESP32 sends data to cloud and the message “Sending data to Blynk Cloud” validates the successful data transfer to cloud. Finally, the ESP32 goes into deep sleep mode for the next hour which is also observed on the serial monitor.

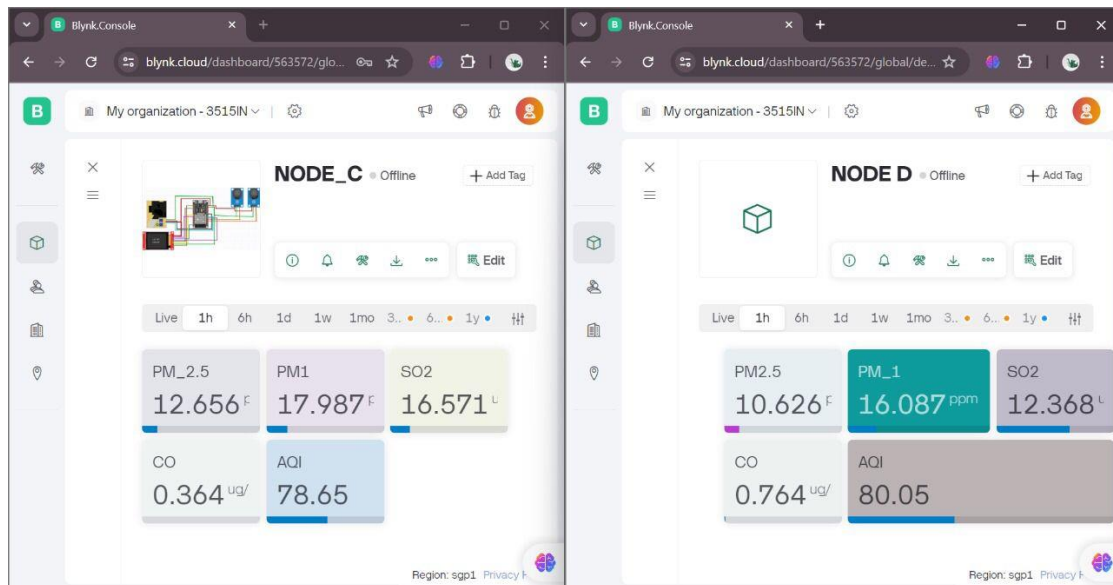


Figure 75: Cloud data visualization in templates

6.3 Predictive Analysis:

6.3.1 Data visualization:

	Datetime	PM_2.5	PM_1	SO2	CO	Total_AQI	Category
0	2024-03-10 18:00:00	141.01	215.23	1.12	9.44	298	Poor
1	2024-03-10 19:00:00	143.23	243.47	1.19	10.54	298	Poor
2	2024-03-10 20:00:00	148.95	261.44	1.18	12.71	300	Poor
3	2024-03-10 21:00:00	148.45	260.60	1.03	13.65	302	Very Poor
4	2024-03-10 22:00:00	124.38	227.54	0.88	13.47	302	Very Poor
	Datetime	PM_2.5	PM_1	SO2	CO	Total_AQI	Category
2713	2024-07-01 19:00:00	20.04	79.83	0.72	7.16	106	Moderate
2714	2024-07-01 20:00:00	20.09	63.18	0.44	5.30	106	Moderate
2715	2024-07-01 21:00:00	21.38	89.86	0.55	5.46	108	Moderate
2716	2024-07-01 22:00:00	23.72	92.88	0.73	6.88	107	Moderate
2717	2024-07-01 23:00:00	34.98	95.58	0.97	7.59	107	Moderate

Figure 76: Data Visualization (part 1)

The output shows a segment of the dataset, which includes the columns for Datetime, PM_2.5, PM_1, SO2, CO, Total_AQI, and Category. The Datetime column represents the timestamp for each data point, while the other columns represent the concentrations of different pollutants and the overall Air Quality Index (AQI). The Category column classifies

the air quality into different levels such as "Poor," "Very Poor," and "Moderate." This initial inspection helps to understand the structure of the data and to verify that the data has been loaded correctly.

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2718 entries, 0 to 2717
Data columns (total 7 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Datetime    2718 non-null  datetime64[ns]
1   PM_2.5      2718 non-null  float64
2   PM_1        2718 non-null  float64
3   SO2         2718 non-null  float64
4   CO          2718 non-null  float64
5   Total_AQI   2718 non-null  int64
6   Category    2718 non-null  object
dtypes: datetime64[ns](1), float64(4), int64(1), object(1)
memory usage: 148.8+ KB
```

Figure 77: Data visualization results (part 2)

The output shows that the dataset contains 2718 entries and 7 columns: Datetime, PM_2.5, PM_1, SO2, CO, Total_AQI, and Category. Each column has 2718 non-null entries, indicating there are no missing values. The data types for the columns are as follows: Datetime is of type datetime64[ns], PM_2.5, PM_1, SO2, and CO are float64, Total_AQI is int64, and Category is object.

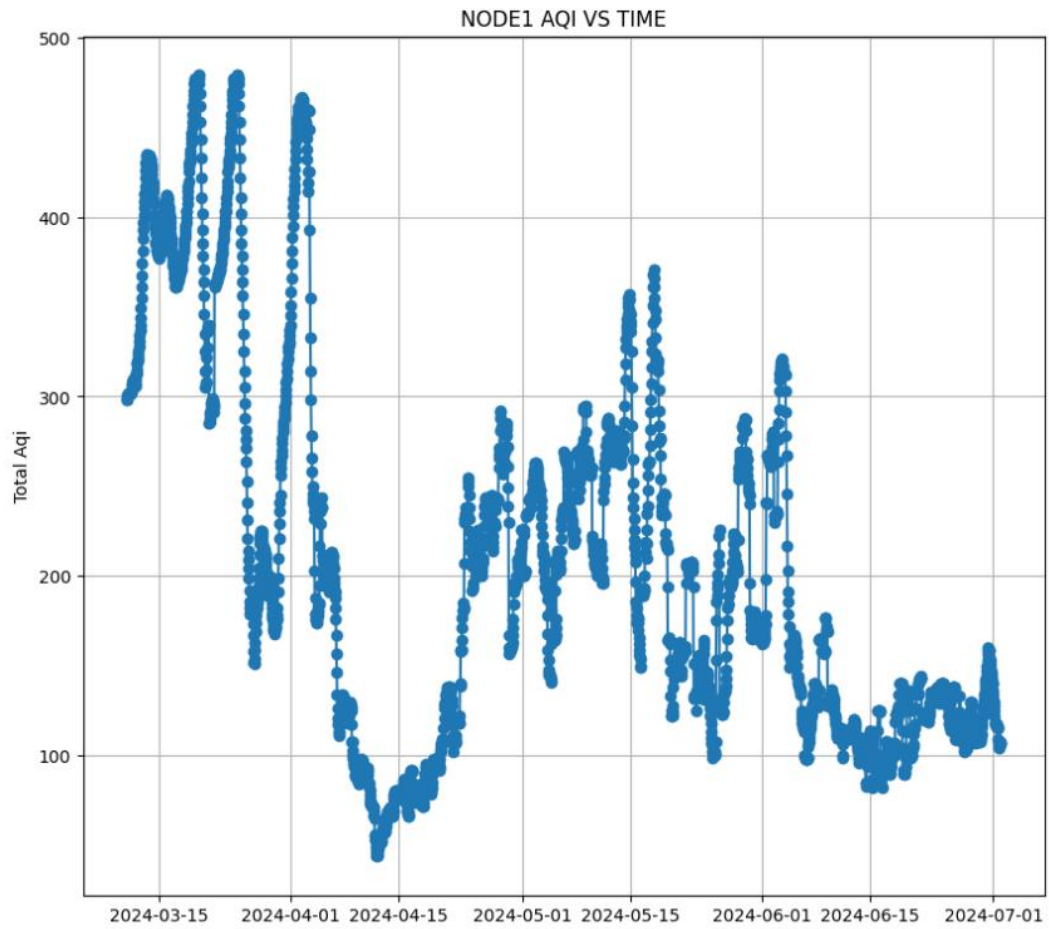


Figure 78: Data visualization results (part 3)

The output graph titled "NODE1 AQI VS TIME" shows the Total_AQI values plotted against the Datetime. The x-axis represents the time, while the y-axis represents the Total AQI. We can see an overall decreasing trend in AQI from the starting month of March till the last month of July. This observation aligns with monthly AQI trends of Pakistan.

6.3.2 Data Correlation:

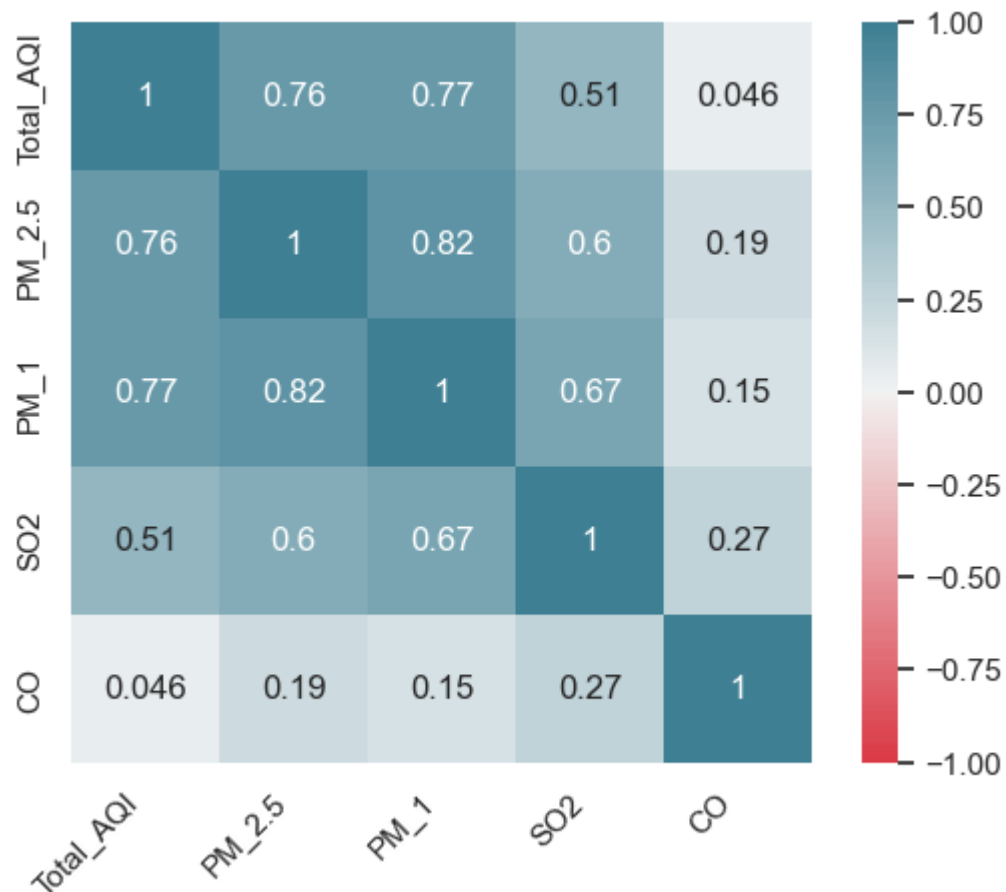


Figure 79: correlation heatmap results

The diagonal from top left to bottom right represents the correlation of each variable with itself. As expected, these values are all 1 (perfect positive correlation). Off-diagonal correlations reveal interesting insights. PM_2.5 and PM_1 exhibit a strong positive correlation (0.82), indicating that as particulate matter concentrations increase, both indicators move together. Total_AQI also correlates well with PM_2.5 (0.76) and PM_1 (0.77). However, carbon monoxide (CO) shows weak positive correlations with other variables (ranging from 0.046 to 0.27)

6.3.3 Data splitting:



Figure 80: Data Split results

The data is split into training and testing sets. The `train_index` is calculated to split the data such that 80% is used for training and 20% for testing.

6.3.4 Predictive modal results:

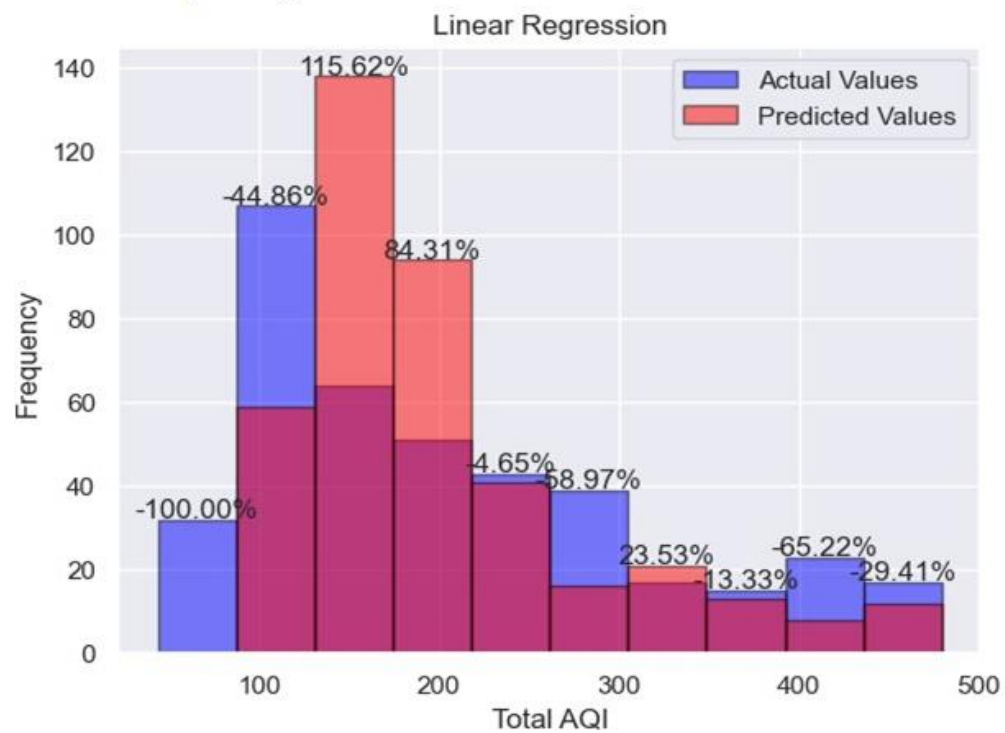


Figure 81: Linear Regression model results

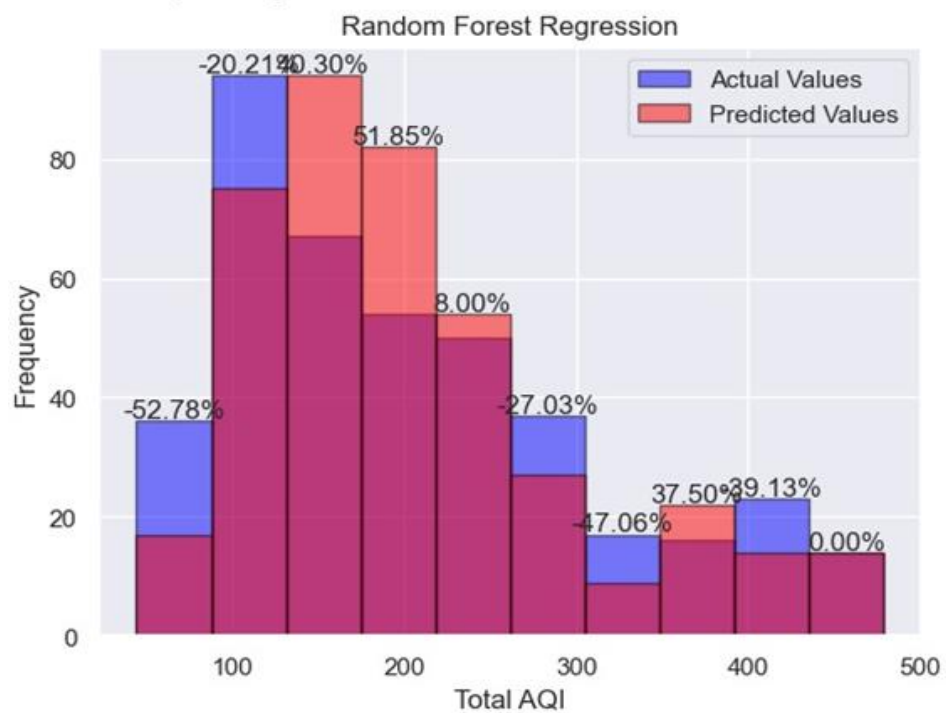


Figure 82: Random Forest Regression model results

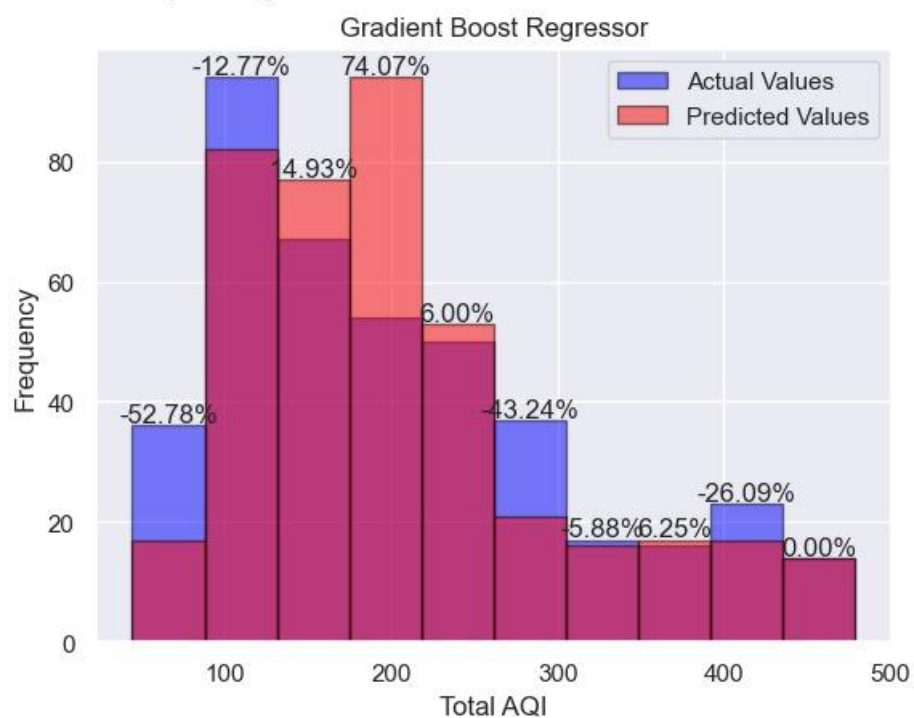


Figure 83: Gradient Boost Regressor Model results

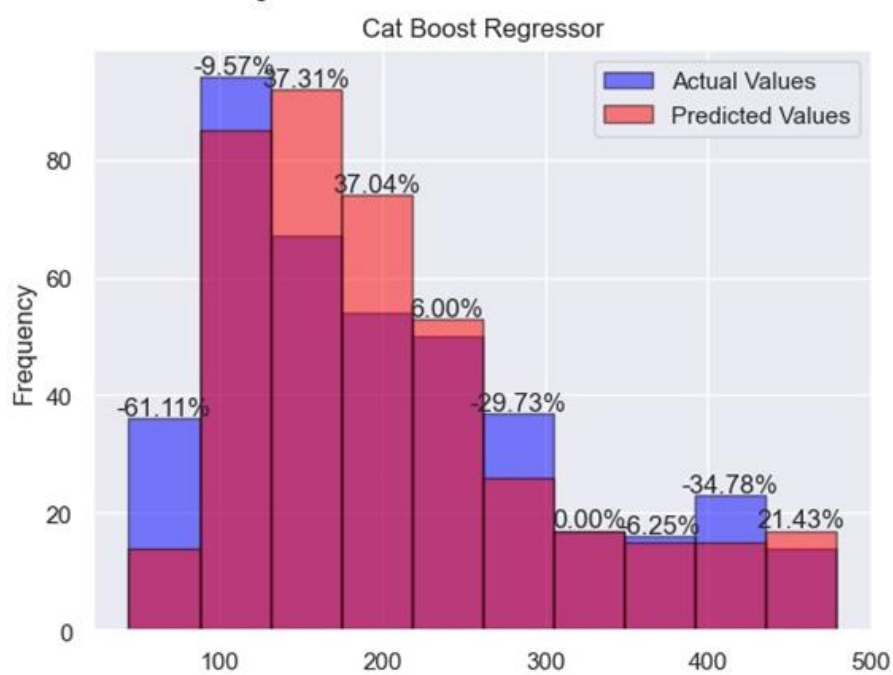


Figure 84: CAT Boost Regressor Model Results

6.3.5 Model test score comparisons:

Tests	Linear Regression	Random Forest	Gradient Boost Regressor	Cat Boost Regressor
Training time	0.00000178 s	0.265625 s	0.609375 s	3.484375 s
Training score	0.653874906114	0.94081400386	0.9808174094	0.95963760
Test score	0.669184843278	0.80082535155	0.7976043971	0.804024957
R2 score	0.669184843278	0.80082535155	0.79760439719	0.8040249570
RMSE	62.57894822397	47.7034497029	48.9481290659	47.3187366460
MAPE	2.810521654935	2.14109907297	2.08569988700	2.053853124489

- **Linear Regression:** Quickest training time but has a moderate performance and higher errors, indicating it's suitable for simple linear relationships.
- **Random Forest:** Good balance of training time and performance with low errors, suitable for more complex datasets.
- **Gradient Boost Regressor:** High accuracy but longer training time, suitable for high predictive performance needs.
- **CatBoost Regressor:** High performance with efficient handling of categorical features but longer training time.

7 CHAPTER 06

7.1 UN-SDGs mapping

Initiating a comprehensive approach to sustainable development, this project aligns closely with four key Sustainable Development Goals (SDGs):

7.1.1 Goal 3: Good Health and Well-being

Through continuous monitoring of air quality and real-time data on pollutants, the project promotes improved health and well-being. Access to precise air quality information empowers individuals and communities to proactively safeguard their health against the adverse effects of air pollution.

7.1.2 Goal 11: Sustainable Cities and Communities

Enhancing urban environmental quality through the monitoring of atmospheric parameters and air quality index (AQI), the project supports sustainable urban development. The data provided enables informed decision-making aimed at enhancing living conditions in cities, such as Islamabad.

7.1.3 Goal 13: Climate Action

The project contributes directly to climate action by monitoring and mitigating air pollution. Understanding pollutant levels and their sources helps stakeholders develop strategies to reduce emissions and mitigate environmental impacts, thereby supporting global efforts to combat climate change.

7.1.4 Goal 17: Partnerships for the Goals

Collaboration among stakeholders, including government agencies, NGOs, and local communities, is crucial for the project's success in air quality monitoring. By fostering

partnerships for environmental data collection, analysis, and decision-making, the project promotes collective action towards achieving sustainable development goals.

8 FUTURE WORK

Looking to the future, this project holds potential for expansion into a larger real-time network for Air Quality Index (AQI) monitoring. Scaling up would involve deploying additional nodes to cover a broader geographical area, allowing for comprehensive AQI assessments across multiple locations simultaneously. The data collected from each node could then be aggregated and analyzed to derive the AQI for larger regions, enhancing our understanding of air quality dynamics on a broader scale.

Improvements in the cloud setup would support the handling of large volumes of AQI data from different locations and seasons. This expanded dataset would enable accurate comparisons over time and geographic regions, providing valuable insights into air quality trends and variations. Such comprehensive data analysis could significantly contribute to environmental awareness and informed decision-making regarding air quality management in Pakistan, ultimately promoting sustainable development and public health initiatives.

9 REFERENCES

- Iqair website, <https://www.iqair.com/pakistan>
- Rawal, R. (2019). Air Quality Monitoring System. International Journal of Computational Science and Engineering, 9(1), 1-9. Retrieved from Research India Publications website: http://www.ripublication.com/ijcse19/ijcsev9n1_01.pdf
- Banerjee, A., Sikdar, B., & Roy, S. (2022). IoT Based Air Pollution Monitoring System. RCC Institute of Information Technology. Retrieved from https://www.rcciit.org/students_projects/projects/ee/2022/GR3.pdf
- Maltare, N. N., & Vahora, S. (2023). Air Quality Index prediction using machine learning for Ahmedabad city. *Government Engineering College*. Retrieved from Science Direct website: <https://www.sciencedirect.com/science/article/pii/S277250812300011X>
- Kumar Sai, K. B., Ramasubbareddy, S., & Luhach, A. K. (2019). IOT BASED AIR QUALITY MONITORING SYSTEM USING MQ135 AND MQ7 WITH MACHINE LEARNING 20(4):599-606. Retrieved from https://www.researchgate.net/publication/337780520_IOT_based_Air_Quality_Monitoring_System_Using_MQ135_and_MQ7_with_Machine_Learning_Analysis
- Ayele, T. W., & Mehta, R. (2018). Air pollution monitoring and prediction using IoT. *2018 Second International Conference on Inventive Communication and Computational Technologies (ICICCT)*. Retrieved from https://www.researchgate.net/publication/327935576_Air_pollution_monitoring_and_prediction_using_IoT